



Graph Deep Factors for Probabilistic Time-series Forecasting

HONGJIE CHEN, Virginia Tech

RYAN A. ROSSI, KANAK MAHADIK, and SUNGCHUL KIM, Adobe Research

HODA ELDARDIRY, Virginia Tech

Effective time-series forecasting methods are of significant importance to solve a broad spectrum of research problems. Deep probabilistic forecasting techniques have recently been proposed for modeling large collections of time-series. However, these techniques explicitly assume either complete independence (local model) or complete dependence (global model) between time-series in the collection. This corresponds to the two extreme cases where every time-series is disconnected from every other time-series in the collection or likewise, that every time-series is related to every other time-series resulting in a completely connected graph. In this work, we propose a deep hybrid probabilistic graph-based forecasting framework called Graph Deep Factors (GraphDF) that goes beyond these two extremes by allowing nodes and their time-series to be connected to others in an arbitrary fashion. GraphDF is a hybrid forecasting framework that consists of a relational global and relational local model. In particular, a relational global model learns complex non-linear time-series patterns globally using the structure of the graph to improve both forecasting accuracy and computational efficiency. Similarly, instead of modeling every time-series independently, a relational local model not only considers its individual time-series but also the time-series of nodes that are connected in the graph. The experiments demonstrate the effectiveness of the proposed deep hybrid graph-based forecasting model compared to the state-of-the-art methods in terms of its forecasting accuracy, runtime, and scalability. Our case study reveals that GraphDF can successfully generate cloud usage forecasts and opportunistically schedule workloads to increase cloud cluster utilization by 47.5% on average. Furthermore, we target addressing the common nature of many time-series forecasting applications where time-series are provided in a streaming version; however, most methods fail to leverage the newly incoming time-series values and result in worse performance over time. In this article, we propose an online incremental learning framework for probabilistic forecasting. The framework is theoretically proven to have lower time and space complexity. The framework can be universally applied to many other machine learning-based methods.

CCS Concepts: • **Computer systems organization** → **Embedded systems**; *Redundancy*; Robotics; • **Networks** → Network reliability;

Additional Key Words and Phrases: Incremental online learning, Graph Neural Network, time-series forecasting

Authors' addresses: H. Chen and H. Eldardiry, Virginia Tech, Blacksburg, Virginia 24061; emails: {jeffchan, hdardiry}@vt.edu; R. A. Rossi, K. Mahadik, and S. Kim, Adobe Research, San Jose, USA; emails: {ryrossi, mahadik, sukim}@adobe.com. Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2023 Association for Computing Machinery.

1556-4681/2023/02-ART26 \$15.00

<https://doi.org/10.1145/3543511>

ACM Reference format:

Hongjie Chen, Ryan A. Rossi, Kanak Mahadik, Sungchul Kim, and Hoda Eldardiry. 2023. Graph Deep Factors for Probabilistic Time-series Forecasting. *ACM Trans. Knowl. Discov. Data.* 17, 2, Article 26 (February 2023), 30 pages.

<https://doi.org/10.1145/3543511>

1 INTRODUCTION

Forecasting is fundamentally important with many applications including the optimization of resources allocation. Time-series forecasting is significantly useful in the business world, as most typically leveraged in stock price prediction and sale outlook forecasting. Recently, forecasting has been utilized for the optimization of resource allocation. For example, accurate forecasting of workload patterns on cloud cluster nodes can help service providers such as AWS or Azure, optimize the resource allocation and scheduling and therefore save money. In cloud resource optimization, the goal is to accurately forecast the resources a service or job will require given CPU and memory usage over time. For this problem, learning and inference must be fast and efficient. For instance, every 5 minutes, we receive new CPU and memory usage measurements, and as soon as we receive them, we need to learn a model and use it to forecast the next h -steps ahead, and then make a decision to scale up/down or not. Additionally, it's important to quantify the prediction uncertainty so that decision-makers can apply different strategies based on the probability of forecast values.

Classical time-series forecasting models such as ARIMA [54] and exponential smoothing models [32] only focus on forecasting individual or small groups of time-series, and hence limiting scalability. In local models, the free parameters are learned independently for each time-series. While these models are sometimes useful, they require a large amount of data for training [38]. Since these models focus on individual time-series, there is often not enough recent data available to make accurate forecasts. As a consequence, they fail to model and extract the mutual connections and dependency across time-series that may help to forecast. Another disadvantage of these models is that they are comparatively simple and they require manual feature engineering and design by domain experts, which is labor-intensive and time-consuming.

Recently, there is a significant increase of data-driven approaches [7, 58] in time-series prediction due to the extensive availability of abundant data from various fields, *e.g.*, shopping behaviors of consumers [31, 65], resource usage optimization for cloud computing [22] and energy consumption [24, 48]. The huge abundance of data makes it necessary to have models that extract limited useful information from big data. At the same time, the intrinsic dependency between time-series also needs to be leveraged for accurate predictions.

In the field of multivariate forecasting, the global models have been studied for decades in econometrics and statistics. In contrast to local models that consider each time-series individually, the free parameters in global models are learned jointly across all time-series in the collection [31, 76]. The assumption behind global models is that all time-series are driven by a small number of latent factors. Among the global models, the deep learning approaches [31, 46, 60] are able to capture complex non-linear time-series patterns. However, in global models, each time-series are equivalently related to any other time-series in the data, which is often violated in practice.

There have also recently been local-global models that attempt to combine the benefits of both [33]. Examples include mixed-effects models [20], where the fix (global) effects describe the whole population while the random (local) effects capture the idiosyncratic behavior of individuals. There are local-global models [66] that combine both types of models for time-series forecasting. However, these models do not solve the disadvantages of ignoring the different relations across

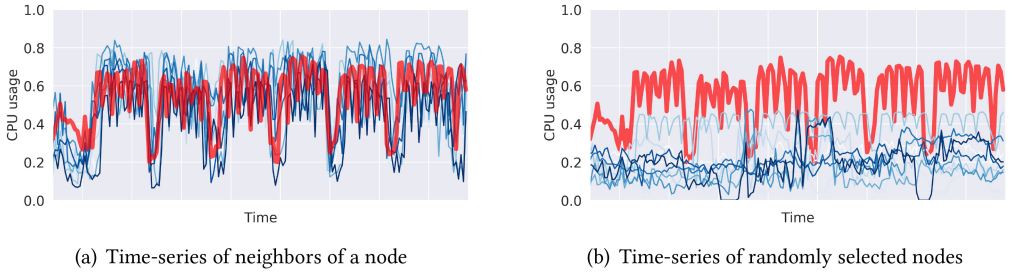


Fig. 1. In (a) the time-series of CPU usage for a node (machine) in the Google workload data shown in **red** and its immediate neighbors in the graph (**blue**) are highly correlated, whereas in (b) the time-series of randomly selected nodes are *significantly different*.

time-series in the global model. Also, the local model is too restricted to model each time-series individually. Thus, we argue that a relational global and relational local model can lead to significantly better forecasting performance with faster training/inference while improving the data efficiency.

In terms of relational time-series forecasting [64], the local models [12, 36] that treat each time-series independently correspond to a graph where each node time-series is not connected to any other nodes and their time-series. Conversely, global models [31, 60, 76] that consider all time-series jointly correspond to a fully connected graph where each node time-series is connected to every other in the same way. These past works all assume time-series are either completely mutually independent or completely dependent. However, these assumptions are often violated in practice as shown in Figure 1 where a node time-series is shown to be dependent on an arbitrary number of other node time-series.

In this work, we propose a deep hybrid *graph-based* probabilistic forecasting model called **Graph Deep Factors (GraphDF)** that allows nodes and their time-series to be dependent (connected) in an arbitrary fashion. GraphDF leverages a *relational global model* that uses the dependencies between time-series in the graph to learn the complex non-linear patterns globally while leveraging a *relational local model* to capture the individual random effects of each time-series locally. GraphDF's relational global model improves the runtime performance and scalability since instead of jointly modeling all time-series together (fully connected graph), which is computationally intensive, GraphDF learns the global latent factors that capture the complex non-linear time-series patterns among the time-series by leveraging only the graph that encodes the dependencies between the time-series. GraphDF serves as a general framework for deep graph-based probabilistic forecasting as many components are completely interchangeable including the relational local and relational global models.

Relational local models use not only the individual time-series but also the neighboring time-series that are one or two hops away in the graph. Thus, the proposed relational local models are more data efficient, especially when considering shorter time-series. For instance, given an individual time-series with a short length (e.g., only six previous values), purely local models would have problems accurately estimating the parameters due to the lack of data points. However, relational local models can better estimate such parameters by leveraging not only the individual time-series but the neighboring dependent time-series that are one or two hops away in the graph. In comparison, relational global models are typically faster and more scalable since they avoid the pairwise dependence assumed by global models via the graph structure. By leveraging the dependencies between time-series encoded in the graph, GraphDF avoids a significant amount of work

that would be required if the time-series are modeled jointly as done in existing state-of-the-art models.

In addition, considering the time-series streaming nature where newly incoming values arrive at each time step, we further propose an **incremental online GraphDF (IOGraphDF)** model that advances GraphDF model tremendously with respect to training runtime. Instead of training a different GraphDF model instance when new values arrive at each time step, only one IOGraphDF model instance is initialized in the first time step, and then the same model instance is modified and updated to accommodate new values over time.

1.1 Main Contributions

We propose a general and extensible deep hybrid graph-based probabilistic forecasting framework called *GraphDF* that is capable of learning complex non-linear time-series patterns globally using the graph time-series data to improve both computational efficiency and forecasting accuracy while learning individual probabilistic models for individual time-series based on their own time-series and the collection of time-series from the immediate neighborhood of the node in the graph. The GraphDF framework is data-driven, fast, scalable for real-time demand forecasting, and highly data efficient.

The state-of-the-art deep probabilistic forecasting methods focus on learning a global model that considers all time-series jointly or a local model learned from each individual time-series independently. In this work, we propose a deep graph-based probabilistic forecasting model that lies in between these two extremes. In particular, we propose a relational global model that learns complex non-linear time-series patterns globally using the structure of the graph to improve both computational efficiency and forecasting performance. Similarly, instead of modeling every time-series independently, we learn a relational local model that not only considers its individual time-series but the time-series of nodes that are connected to an individual node in the graph.

Furthermore, the proposed GraphDF framework applies to a significantly larger class of problems, which includes prior work as a special case. In particular, GraphDF naturally generalizes many existing models including those based purely on local and global models, or a combination of both. This is due to its flexibility to interpolate between purely non-relational models (either local, global, or both) and relational models that leverage the graph structure encoding the dependencies between the different time-series. The experiments demonstrate the effectiveness of the proposed deep graph-based probabilistic forecasting model in terms of its forecasting performance, runtime, and scalability.

Finally, we extend GraphDF to meet the incremental online scheme and derive the IOGraphDF model, which converges over a timespan to yield approximately accurate predictions as GraphDF, but takes a much shorter time to train and update.

2 RELATED WORK

Classical Time-series Forecasting. A vast variety of forecasting approaches have been developed for its wide applications and usage in various domain [5, 15, 30, 55, 77]. Classical time-series models including **autoregressive integrated moving average (ARIMA)** and exponential smoothing [32, 54] have demonstrated a huge success in univariate time-series prediction, however, they fail to extract the non-linear relationships across time-series. Besides that, they are incapable of modeling the exogenous values, which usually help to forecast. By contrast, multivariate time-series prediction [10, 65, 71] takes the advantage of modeling the inter-dependencies across time-series to improve the prediction accuracy. One example of multivariate time-series models is **vector autoregression (VAR)** [69] which is commonly considered a generalization of autoregressive model. However, VAR treats the relationships across time-series equivalently without

difference, which is unrealistic. Deb et al. [21] summarized nine classical methods for forecasting energy usage including **artificial neural network** (ANN), **support vector machine** (SVR), and others.

Deep Learning-based Time-series Forecasting. In recent years, advances in deep learning have led to substantial improvements [23, 35, 51] in time-series prediction, among which **recurrent neural networks** (RNNs) received a great extent of popularity [1, 11, 39, 83] due to their significant accuracy in predictions and flexibility to model the non-linear relationships [7]. As prominent examples of RNN model, the **long short-term memory** (LSTM) units [6, 41] and the **gated recurrent units** (GRU) [18] are broadly adopted for their competence to overcome the vanishing gradient problem. Based on the LSTM and GRU architectures, sequence-to-sequence models [4, 50, 70] are developed to allow predictions for a modest number of horizons [26, 76].

While typical RNN models target univariate time-series prediction, a substantial amount of efforts have been made to share information across time-series to model the highly non-linear interdependencies and thus improve the forecast accuracy. For instance, Qin et al. [59] proposed a dual-stage attention-based RNN model. Huang et al. [42] introduced a dual-attention mechanism for dynamic-period or non-periodic multivariate time-series forecasting. These methods assume all time-series are equally related to each other, which can be seen as having a fully connected graph.

While earlier work focused on *point forecasting* which aims at predicting optimal expected values, there is an increasing interest in *probabilistic forecasting* models [40, 47, 53, 62, 78]. Probabilistic models yield prediction as distributions and have the advantage of uncertainty estimates, which are important for downstream decision-making. Some recent probabilistic models are proposed in the multivariate manner, for example, Salinas et al. [31] proposed a probabilistic forecasting model that jointly learns a global model from all available time-series. Wang et al. proposed DF [75], a hybrid global-local model that assumes time-series are determined by shared factors as well as individual randomness. These methods indiscriminately model mutual dependence between time-series. Hence, they imply a strong and unrealistic assumption that all time-series are pairwise related to one another in a uniformly equivalent way.

In contrast, we propose a hybrid deep graph-based probabilistic forecasting framework that leverages a relational graph global component that learns the complex non-linear time-series patterns in the large collection of relational time-series data and a relational local component that handles uncertainty by learning a probabilistic forecasting model for every individual node in the graph that not only considers the time-series of the individual node, but also the time-series of nodes directly connected in the graph. The relational global component of the proposed GraphDF framework leverages the graph time-series data, leading to a significant improvement in the time-efficiency, scalability, and most importantly, the forecasting accuracy of our model compared to the state-of-the-art DF model. Conversely, the relational local model of GraphDF has the advantage of improving both forecasting accuracy and data efficiency.

Graph-based Models. Modeling the unique relations to each individual time-series from others naturally leads us to graph models. For instance, **Graph Neural Network** (GNN) [14, 43, 44, 84] has recently shown great success in extracting the information across nodes. Moreover, the combination of GNNs and RNNs [34, 74] allows the injection of dynamism of the pairwise non-linear relationship across time-series. As an early work, Seo et al. [67] introduced **graph convolutional recurrent network** (GCRN) to predict structured sequential data. Other recent work is mostly limited in spatio-temporal study, such as traffic prediction [82] and ride-hailing demand forecasting [80, 81]. All these methods are incorporated with a graph structure. Besides, these methods are not probabilistic models and they fail to deliver uncertainty estimates.

Resource usage Prediction. Researchers and engineers put great efforts on resource provisioning and load prediction in cloud-scale systems [8, 56, 72]. Early work mainly utilized the traditional state space models such as ARIMA [13, 85]. More recent work covers both traditional methods [13, 85], machine learning approaches [19] such as K-nearest neighbors [29, 68] and linear regression [28, 79], and RNN-based methods [17, 27, 45]. However, none of these methods leverages a graph to model the relationships between nodes.

Prediction of Streaming Data. In many applications, data values are not given beforehand but instead arrive continuously with an equivalent time gap between arrivals. Early work on prediction in this kind of scenario includes modifying ARIMA models to an online manner [3, 52], predicting with kernel-based methods [63], and efforts on elastic resource scaling to reduce cloud system operating cost [9, 68]. More recent work leverages deep learning on streaming data. For instance, Vrabecová et al. [73] proposed a stream change detection method to identify the ongoing changes or concept drifts in the power meter data. Guo et al. [37] proposed an adaptive gradient learning method that aims at minimizing impacts from outliers as well as leveraging the local features, but this work is solely based on RNN and only targets univariate time-series prediction. A more recent RNN-based work [25] targets finding mismatch of temporal distribution between periods of time-series. However, these models do not leverage graph structures or the inter-correlations between time-series for forecasting. By contrast, our proposed work is graph-based and has the advantage of forecasting accuracy and runtime efficiency.

3 GRAPH DEEP FACTORS

In this section, we describe a general and extensible framework called GraphDF. It is capable of learning complex non-linear time-series patterns globally using the graph time-series data to improve both computational efficiency and performance while learning probabilistic models for each individual time-series based on their own time-series and the collection of related time-series from the neighborhood of the node in the graph. The GraphDF framework is data-driven, flexible, accurate, and scalable for large collections of multi-dimensional time-series data.

3.1 Problem Formulation

We first introduce the deep graph-based probabilistic forecasting problem. Notably, this is the first hybrid deep graph-based probabilistic forecasting framework. The framework is comprised of a graph relational global component (described in Section 3.3) that learns the complex non-linear time-series patterns in the large collection of graph-based time-series data and a relational local component (Section 3.4) that handles uncertainty by learning a probabilistic forecasting model for every individual node in the graph that not only considers the time-series of the individual node, but also the time-series of nodes directly connected in the graph. This has the advantage of improving both forecasting accuracy and data efficiency.

The proposed framework solves the following graph-based time-series forecasting problem. Let $G = (V, E, \mathcal{X}, \mathcal{Z})$ denote the graph model where V is the set of nodes, E is the set of edges, and $\mathcal{X} = \{\mathbf{X}^{(i)}\}_{i=1}^N$ is the set of covariate time-series associated with the N nodes in G where $\mathbf{X}^{(i)} \in \mathbb{R}^{D \times T}$ is the covariate time-series data associated with node i . Hence, each node is associated with D different covariate time-series. Furthermore, $\mathcal{Z} = \{\mathbf{z}^{(i)}\}_{i=1}^N$ is the set of time-series associated with the N nodes in G . The N nodes can be connected in an arbitrary fashion that reflects the dependence between nodes. Two nodes i and j that contain an edge $(i, j) \in E$ in the graph G encodes an explicit dependency between the time-series data of node i and j . Intuitively, using these explicit dependencies encoded in G can lead to more accurate forecasts as shown in Figure 1. Furthermore, let $\mathbf{z}_{1:T}^{(i)}$ denote a univariate time-series for node i in the graph where $\mathbf{z}_{1:T}^{(i)} = [z_1^{(i)} \cdots z_T^{(i)}] \in \mathbb{R}^T$.

and $z_t^{(i)} \in \mathbb{R}$. In addition, each node i in the graph G also has D covariate time-series, $\mathbf{X}^{(i)} \in \mathbb{R}^{D \times T}$ where $\mathbf{X}_{:,t}^{(i)} \in \mathbb{R}^D$ (or $\mathbf{x}_t^{(i)} \in \mathbb{R}^D$) represents the D covariate values at time step t for node i . We also denote $\mathbf{A} \in \mathbb{R}^{N \times N}$ as the sparse adjacency matrix of the graph G where $N = |V|$ is the number of nodes. If $(i, j) \in E$, then A_{ij} denotes the weight of the edge (dependency) between node i and j , and $A_{ij} = 0$ when $(i, j) \notin E$ otherwise.

We denote the unknown model parameters as Φ . Our goal is to learn a generative probabilistic forecasting model described by Φ that gives the (joint) distribution on target values in the future horizon τ :

$$\mathbb{P}\left(\left\{z_{T+1:T+\tau}^{(i)}\right\}_{i=1}^N \mid \mathbf{A}, \left\{z_{1:T}, \mathbf{X}_{:,1:T+\tau}^{(i)}\right\}_{i=1}^N; \Phi\right). \quad (1)$$

Hence, solving Equation (1) gives the joint probability distribution over future values given all covariates and past observations along with the graph structure represented by $\mathbf{A}$ that encodes the explicit dependencies between the N nodes and their corresponding time-series $\{z^{(i)}, \mathbf{X}^{(i)}\}_{i=1}^N$.

Graph Construction. For each dataset, we derive a graph where each node represents a machine with one or more time-series associated with it, and each edge represents the similarity between the node time-series i and j . The constructed graph encodes the dependency information between nodes. In this work, we estimate the edge weights using the **radial basis function (RBF)** kernel with the previous time-series observations as $K(\mathbf{z}_i, \mathbf{z}_j) = \exp(-\frac{\|\mathbf{z}_i - \mathbf{z}_j\|^2}{2\ell^2})$, where ℓ is the length scale of the kernel.

3.2 Framework Overview

The GraphDF framework aims at learning a parametric distribution to predict future values. In GraphDF, each node i and its time-series $z_t^{(i)}, \forall t = 1, 2, \dots$ can be connected to other nodes and their time-series in an arbitrary fashion, which is encoded in the graph G . These connections represent explicit dependencies or correlations between the time-series of the nodes. Furthermore, we also assume that each node i and their time-series $\mathbf{z}_{1:t}^{(i)}$ are governed by two key components including (1) a relational global model (Section 3.3), and (2) a relational local random effect model (Section 3.4). As such, GraphDF is a hybrid forecasting framework. Both the relational global component and relational local component of our framework leverage the graph via the specific underlying model used for each component.

In the relational global component of GraphDF, we assume there are K latent relational global factors that determine the fixed effect to each node and their time-series. Specifically, the relational global model consists of an approach that leverages the adjacency matrix $\mathbf{A}$ of the graph G and $\{\mathbf{X}_{:,1:t}, \mathbf{z}_{1:t-1}^{(i)}\}$ for learning the K relational global factors that capture the relational non-linear time-series patterns in the graph-based time-series data,

$$\text{relational global factors: } s_k(\cdot) = \text{GCRN}_k(\cdot), \quad k = 1, \dots, K, \quad (2)$$

where $s_k(\cdot)$, $k = 1, 2, \dots, K$ are the K relational global factors that govern the underlying graph-based time-series data of all nodes in G . In Equation (2), we learn the relational global factors using a GCRN [67]; however, GraphDF is flexible for use with any other arbitrary deep time-series model such as DCRNN, among many other possibilities. These are then used to obtain the relational global fixed effects function $c^{(i)}$ for node i as follows:

$$\text{fixed effect: } c^{(i)}(\cdot) = \sum_{k=1}^K w_{i,k} \cdot s_k(\cdot), \quad (3)$$

where $w_{i,k}$ represents the K -dimensional embedding for node i . Therefore, the final relational non-random fixed effect for node i is simply a linear combination of the K global factors and the

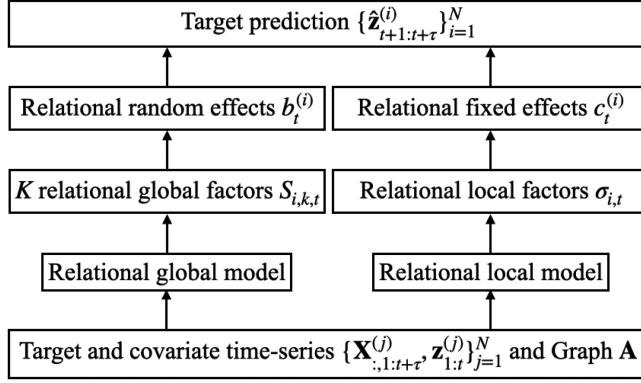


Fig. 2. An overview of GraphDF framework.

embedding $\mathbf{w}_i \in \mathbb{R}^K$ for node i . Now we use a relational local model discussed in Section 3.4 to obtain the local random effects for each node i . More formally, we define the *relational local random effects* function $b^{(i)}$ for a node i in the graph G as

$$\text{relational local random effect: } b^{(i)}(\cdot) \sim \mathcal{R}_i, \quad i = 1, \dots, N, \quad (4)$$

where $\mathcal{R}_i$ can be any relational probabilistic time-series model. To compute $\mathbb{P}(\mathbf{z}_{1:t}^i | \mathcal{R}_i)$ efficiently, we ensure $b_t^{(i)}$ obeys a normal distribution and thus can be derived fast. The *relational latent function* of node i denoted as $v^{(i)}$ is then defined as

$$\text{latent function: } v^{(i)}(\cdot) = c^{(i)}(\cdot) + b^{(i)}(\cdot), \quad (5)$$

where $c^{(i)}$ is the relational fixed effect of node i and $b^{(i)}$ is the relational local random effect for node i . Hence, the relational latent function of node i is simply a linear combination of the relational fixed effect $c^{(i)}$ from Equation (3) and its local relational random effect $b^{(i)}$ from Equation (4). Then

$$\text{emission: } z_t^{(i)} \sim \mathbb{P}\left(z_t^{(i)} \mid v^{(i)}(\mathbf{A}, \{\mathbf{X}_{:,1:t}^{(j)}, \mathbf{z}_{1:t-1}^{(j)}\}_{j=1}^N)\right), \quad (6)$$

where the observation model $\mathbb{P}$ can be any parametric distribution. For instance, $\mathbb{P}$ can be Gaussian, Poisson, Negative Binomial, among others.

The GraphDF framework is defined in Equations (2)–(6). All the functions $s_k(\cdot)$, $b^{(i)}(\cdot)$, $v^{(i)}(\cdot)$ take past observations and covariates $\{\mathbf{z}_{1:t-1}^{(j)}, \mathbf{X}_{:,1:t}^{(j)}\}_{j=1}^N$, as well as the graph structure in the form of adjacency matrix $\mathbf{A}$ as inputs. We define $\mathbf{w}_i = [w_{i,1} \cdots w_{i,k} \cdots w_{i,K}] \in \mathbb{R}^K$ as the K -dimension embedding for time-series $\mathbf{z}^{(i)}$ where $w_{i,k} \in \mathbb{R}$ is the weight of the k -th factor for node i . An overview of the GraphDF framework is depicted in Figure 2.

3.3 Relational Global Model

The relational global model learns K relational global factors from all time-series by a graph-based model. These relational global factors are considered as the driving latent factors. After the relational global factors are derived from the model, they are then used in a linear combination with weights given by embeddings for each time-series $\mathbf{w}_i$, as shown in Equation (3).

3.3.1 Learning Relational Global Factors via GCRN. We first show how GCRN [67] can be modified for learning relational global factors in GraphDF. Let $\mathbf{x}_t^{(i)} \in \mathbb{R}^D$ denote the D covariates of node i at time step t . Now, we define the input temporal features of the relational global factor

component of the graph G as

$$\mathbf{Y}_t = \begin{bmatrix} \mathbf{z}_{t-1}^{(1)} & \mathbf{x}_t^{(1)} \\ \vdots & \vdots \\ \mathbf{z}_{t-1}^{(N)} & \mathbf{x}_t^{(N)} \end{bmatrix} \in \mathbb{R}^{N \times P}, \quad (7)$$

where $P = D + 1$ for simplicity. We refer to $\mathbf{Y}_t$ as a time-series graph signal. The aggregation of information from other nodes is performed by a graph convolution operation defined as the multiplication of a temporal graph signal with a filter g_θ . Given input features $\mathbf{Y}_t$, the graph convolution operation is denoted as $f_{\star_{\mathcal{G}}} \Theta$ with respect to the graph G and parameters θ :

$$f_{\star_{\mathcal{G}}} \Theta(\mathbf{Y}_t) = g_\theta(\mathbf{L})\mathbf{Y}_t \quad (8)$$

$$= \mathbf{U}g_\theta(\Lambda)\mathbf{U}^T\mathbf{Y}_t \in \mathbb{R}^{N \times P}, \quad (9)$$

where $\mathbf{L} = \mathbf{I} - \mathbf{D}^{-\frac{1}{2}}\mathbf{A}\mathbf{D}^{-\frac{1}{2}}$ is the normalized Laplacian matrix of the adjacency matrix, $\mathbf{I} \in \mathbb{R}^{N \times N}$ is an identity matrix. $D_{ii} = \sum_j A_{ij}$ is the diagonal weighted degree matrix. $\mathbf{L} = \mathbf{U}\Lambda\mathbf{U}^T$ is the eigenvalue decomposition. $\mathbf{U}$ is the matrix composed of eigenvectors by order of eigenvalues of $\mathbf{L}$, and Λ is the diagonal matrix of eigenvalues of $\mathbf{L}$. $g_\theta(\Lambda) = \text{diag}(\boldsymbol{\theta})$ denotes a filter parameterized by the coefficients $\boldsymbol{\theta} \in \mathbb{R}^N$ in the Fourier domain. Directly applying Equation (9) is computationally expensive due to the matrix multiplication and the eigen-decomposition of $\mathbf{L}$. To accelerate the computation speed, the Chebyshev polynomial approximation up to a selected order $L - 1$ is

$$g_\theta(\mathbf{L}) = \sum_{l=0}^{L-1} \theta_l T_l(\tilde{\mathbf{L}}), \quad (10)$$

where $\boldsymbol{\theta} = [\theta_0 \cdots \theta_{L-1}] \in \mathbb{R}^L$ in Equation (10) is the Chebyshev coefficients vector. Importantly, $T_l(\tilde{\mathbf{L}}) = 2\tilde{\mathbf{L}}T_{l-1}(\tilde{\mathbf{L}}) - T_{l-2}(\tilde{\mathbf{L}})$ is recursively computed with the scaled Laplacian $\tilde{\mathbf{L}} = 2\mathbf{L}/\lambda_{\max} - \mathbf{I} \in \mathbb{R}^{N \times N}$, and starting values $T_0 = 1$ and $T_1 = \tilde{\mathbf{L}}$. The Chebyshev polynomial approximation improves the time complexity to linear in the number of edges $O(L|E|)$, *i.e.*, number of dependencies between the multi-dimensional node time-series. The order L controls the local neighborhood time-series that are used for learning the relational global factors, *i.e.*, a node's multi-dimensional time-series only depends on neighboring node time-series that are at maximum L hops away in the graph G .

Let $\Theta \in \mathbb{R}^{P \times Q \times L}$ be a tensor of parameters that map the dimension P of input to the dimension Q of output:

$$\mathbf{H}_{:,q} = \tanh \left[\sum_{p=1}^P f_{\star_{\mathcal{G}}} \Theta(\mathbf{Y}_t, :, p) \right], \text{ for } q \in 1 \dots Q. \quad (11)$$

The relational global component integrates the temporal dependence and relational dependence among nodes with the graph convolution,

$$\mathbf{I}_t = \sigma(\Theta_I \star_{\mathcal{G}} [\mathbf{Y}_t, \mathbf{H}_{t-1}] + \mathbf{W}_I \odot \mathbf{C}_{t-1} + \mathbf{b}_I), \quad (12)$$

$$\mathbf{F}_t = \sigma(\Theta_F \star_{\mathcal{G}} [\mathbf{Y}_t, \mathbf{H}_{t-1}] + \mathbf{W}_F \odot \mathbf{C}_{t-1} + \mathbf{b}_F), \quad (13)$$

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tanh(\Theta_C \star_{\mathcal{G}} [\mathbf{Y}_t, \mathbf{H}_{t-1}] + \mathbf{b}_C), \quad (14)$$

$$\mathbf{O}_t = \sigma(\Theta_O \star_{\mathcal{G}} [\mathbf{Y}_t, \mathbf{H}_{t-1}] + \mathbf{W}_O \odot \mathbf{C}_t + \mathbf{b}_O), \quad (15)$$

$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t), \quad (16)$$

where $\mathbf{I}_t, \mathbf{F}_t, \mathbf{O}_t \in \mathbb{R}^{N \times Q}$ are the input, forget, and output gate in the LSTM structure. Q is the number of hidden units, $\mathbf{W}_I, \mathbf{W}_F, \mathbf{W}_O \in \mathbb{R}^{N \times Q}$ and $\mathbf{b}_I, \mathbf{b}_F, \mathbf{b}_C, \mathbf{b}_O \in \mathbb{R}^Q$ are weights and bias parameters, $\Theta_I, \Theta_F, \Theta_C, \Theta_O \in \mathbb{R}^{P \times Q}$ are parameters corresponding to different filters.

The hidden state $\mathbf{H}_t \in \mathbb{R}^{N \times Q}$ encodes the observation information from $\mathbf{H}_{t-1}$ and $\mathbf{Y}_t$, as well as the relations across nodes through the graph convolution described by $\Theta \star_{\mathcal{G}} (\cdot)$ in Equation (8). From hidden state $\mathbf{H}_t$, we derive the value of K relational global factors at time step t as $\mathbf{S}_t \in \mathbb{R}^{N \times K}$ through a fully connected layer,

$$\mathbf{S}_t = \mathbf{H}_t \mathbf{W} + \mathbf{b}, \quad (17)$$

where $\mathbf{W} \in \mathbb{R}^{Q \times K}$ and $\mathbf{b} \in \mathbb{R}^K$ are the weight matrix and bias vector trained in the model (for the K relational global factors), respectively. The relational global factors $\mathbf{S}_t$ is derived from the Equation (17) that capture the complex non-linear time-series patterns between the different time-series globally.

Finally, the fixed effect at time t is derived for each node i as a weighted sum with the embedding $\mathbf{w}_i \in \mathbb{R}^K$ and the relational global factors $\mathbf{S}_t$, as

$$c_t^{(i)}(\cdot) = \sum_{k=1}^K w_{i,k} \cdot S_{i,k,t}. \quad (18)$$

The embedding $\mathbf{w}_i$ represents the weighted contribution that each relational factor has on node i .

3.3.2 Learning Relational Global Factors via DCRNN. For the relational global component of GraphDF, we can also leverage DCRNN [49]. Different from the GCRN model, the original DCRNN leverages a diffusion convolution operation and a GRU structure for learning the relational global factors of GraphDF.

Given the time-series graph signal, $\mathbf{Y}_t \in \mathbb{R}^{N \times P}$ with N nodes, the diffusion convolution with respect to the graph-based time-series is defined as

$$f_{\star_{\mathcal{G}}} \Theta(\mathbf{Y}_t) = \sum_{l=0}^{L-1} (\theta_l \tilde{\mathbf{A}}^l) \mathbf{Y}_t, \quad (19)$$

where $\tilde{\mathbf{A}} = \mathbf{D}^{-1} \mathbf{A}$ is the normalized adjacency matrix of the graph G that captures the explicit weighted dependencies between the multi-dimensional time-series of the nodes. The Chebyshev polynomial approximation is used similarly to Equation (10).

The relational global factors are learned using the graph diffusion convolution combined with GRU enabling them to be carried forward over time using the graph structure,

$$\mathbf{R}_t = \sigma(\Theta_R \star_{\mathcal{G}} [\mathbf{Y}_t, \mathbf{H}_{t-1}] + \mathbf{b}_R), \quad (20)$$

$$\mathbf{U}_t = \sigma(\Theta_U \star_{\mathcal{G}} [\mathbf{Y}_t, \mathbf{H}_{t-1}] + \mathbf{b}_U), \quad (21)$$

$$\mathbf{C}_t = \tanh(\Theta_C \star_{\mathcal{G}} [\mathbf{Y}_t, (\mathbf{R}_t \odot \mathbf{H}_{t-1})] + \mathbf{b}_C), \quad (22)$$

$$\mathbf{H}_t = \mathbf{U}_t \odot \mathbf{H}_{t-1} + (1 - \mathbf{U}_t) \odot \mathbf{C}_t, \quad (23)$$

where $\mathbf{H}_t \in \mathbb{R}^{N \times Q}$ denotes the hidden state of the model at time step t , Q is the number of hidden units, $\mathbf{R}_t, \mathbf{U}_t \in \mathbb{R}^{N \times Q}$ are called as reset gate and update gate at time t , respectively. $\Theta_R, \Theta_U, \Theta_C \in \mathbb{R}^L$ denote the parameters corresponding to different filters.

With the hidden state $\mathbf{H}_t$ in Equation (23), the fixed effect is derived from DCRNN similarly to Equations (17) and (18). Compared to the previous GCRN that we adapted for the relational global component, DCRNN is more computationally efficient due to the GRU structure it uses.

3.4 Relational Local Model

The (stochastic) relational local component handles uncertainty by learning a probabilistic forecasting model for every individual node in the graph G that not only considers the time-series of

the individual node but also the time-series of nodes directly connected. This has the advantage of improving both forecasting accuracy and data efficiency.

The random effects in the relational local model represent the local fluctuations of the individual node time-series. The relational local random effect for each node time-series $b^{(i)}$ is sampled from the relational local model $\mathcal{R}_i$, as shown in Equation (4). For $\mathcal{R}_i$, we choose the Gaussian distribution as the likelihood function for sampling, but other parametric distributions such as Student-t or Gamma distributions are also possible. Compared to the relational global component of GraphDF from Section 3.3 that uses the entire graph G along with all the node multi-dimensional time-series to learn K global factors that capture the most important non-linear time-series patterns in the graph-based time-series data, the relational local component focuses on modeling an individual node $i \in V$ and therefore leverages only the time-series of node i and the set of highly correlated time-series from its immediate local neighborhood Γ_i . Hence, $\{z^{(j)}, \mathbf{X}^{(j)}\}, j \in \Gamma_i$. Intuitively, the relational local component of GraphDF achieves better data efficiency by leveraging the highly correlated neighboring time-series along with its own time-series. This allows GraphDF to make more accurate forecasts further in the future with less training data. We now introduce probabilistic GCRN and probabilistic DCRNN model that can be used as the stochastic relational local component in GraphDF.

3.4.1 Estimating Uncertainties via Probabilistic GCRN. In this section, we propose a relational local probabilistic GCRN model for use with the GraphDF framework. In contrast to the relational global model in Section 3.3, the relational local model focuses on learning an individual local model for each individual node based on its own multi-dimensional time-series data as well as the nodes neighboring it. This enables us to model the local fluctuations of the individual multi-dimensional time-series data of each node.

Compared to RNN, the benefits of the proposed probabilistic GCRN model in the local component is that it not only models the sequential nature of the data but also exploits the graph structure by using the surrounding nodes to learn a more accurate model for each individual node in G . This is an ideal property for we assume the fluctuations of each node are related to those of other connected nodes in the ℓ -localized neighborhood, which was shown to be the case in Figure 1.

For simplicity, let $C = \Gamma_i$ denote the set of neighbors of a node i in the graph G . Note that C can be thought of as the set of related neighbors of node i , which may be the immediate 1-hop neighbors, or more generally, the ℓ -hop neighbors of i . Recall that we define $\mathbf{x}_t^{(i)} \in \mathbb{R}^D$ as the D covariates of node i at time t . Then, we define $\mathbf{X}_t^C$ as an $|C| \times D$ matrix consisting of the covariates of all the neighboring nodes $j \in C$ of node i .

$$\mathbf{X}_t^{(C)} = \begin{bmatrix} \mathbf{x}_t^{(C_1)} & \mathbf{x}_t^{(C_2)} & \dots & \mathbf{x}_t^{(C_{|C|})} \end{bmatrix}^\top, \quad (24)$$

where C_j denotes the j th neighbor. Now, we define the input temporal features of the relational local model for node i as

$$\mathbf{Y}_t^{(i)} = \begin{bmatrix} z_{t-1}^{(i)} & \mathbf{x}_t^{(i)\top} \\ z_{t-1}^{(C)} & \mathbf{X}_t^{(C)} \end{bmatrix}. \quad (25)$$

Let $\mathbf{L}^{(i)} \in \mathbb{R}^{(|C|+1) \times (|C|+1)}$ denote the submatrix of Laplacian matrix $\mathbf{L}$ that consist of rows and columns corresponding to node i and its neighbors C . For each node i , we derive the relational local random effect using its past observations and covariates of the node i and those of its neighbors

through the graph convolution with respect to $\mathbf{L}^{(i)}$.

$$\begin{aligned}\mathbf{I}_t^{(i)} &= \sigma \left(\Theta_I^{(i)} \star_{\mathcal{G}} [\mathbf{Y}_t^{(i)}, \mathbf{H}_{t-1}^{(i)}] + \mathbf{W}_I^{(i)} \odot \mathbf{C}_{t-1}^{(i)} + \mathbf{b}_I^{(i)} \right), \\ \mathbf{F}_t^{(i)} &= \sigma \left(\Theta_F^{(i)} \star_{\mathcal{G}} [\mathbf{Y}_t^{(i)}, \mathbf{H}_{t-1}^{(i)}] + \mathbf{W}_F^{(i)} \odot \mathbf{C}_{t-1}^{(i)} + \mathbf{b}_F^{(i)} \right), \\ \mathbf{C}_t^{(i)} &= \mathbf{F}_t^{(i)} \odot \mathbf{C}_{t-1}^{(i)} + \mathbf{I}_t^{(i)} \odot \tanh \left(\Theta_C^{(i)} \star_{\mathcal{G}} [\mathbf{Y}_t^{(i)}, \mathbf{H}_{t-1}^{(i)}] + \mathbf{b}_C^{(i)} \right), \\ \mathbf{O}_t^{(i)} &= \sigma \left(\Theta_O^{(i)} \star_{\mathcal{G}} [\mathbf{Y}_t^{(i)}, \mathbf{H}_{t-1}^{(i)}] + \mathbf{W}_O^{(i)} \odot \mathbf{C}_t^{(i)} + \mathbf{b}_O^{(i)} \right), \\ \mathbf{H}_t^{(i)} &= \mathbf{O}_t^{(i)} \odot \tanh \left(\mathbf{C}_t^{(i)} \right),\end{aligned}$$

where $\Theta_I^{(i)}, \Theta_F^{(i)}, \Theta_C^{(i)}, \Theta_O^{(i)} \in \mathbb{R}^{P \times R}$ denote the parameters corresponding to different filters of the relational local model, R is the number of hidden units in the relational local model, and recall $P = D + 1$. Furthermore, $\mathbf{H}_t^{(i)} \in \mathbb{R}^{(|C|+1) \times R}$ is the hidden state for node i and its neighbors Γ_i . $\mathbf{W}_I^{(i)}, \mathbf{W}_F^{(i)}, \mathbf{W}_O^{(i)} \in \mathbb{R}^{(|C|+1) \times R}$ are weight matrix parameters and $\mathbf{b}_I^{(i)}, \mathbf{b}_F^{(i)}, \mathbf{b}_C^{(i)}, \mathbf{b}_O^{(i)} \in \mathbb{R}^R$ are bias vector parameters. Note in the above formulation, we assume $\ell = 1$, hence, only the immediate 1-hop neighbors are used.

From the hidden state $\mathbf{H}_t^{(i)}$, we only take the row corresponding to node i to derive the relational local random effect for node i . We denote the value as $\mathbf{h}_t^{(i)} \in \mathbb{R}^R$, and apply a fully connected layer with a softplus activation function to aggregate the hidden units,

$$\sigma_{i,t} = \log \left(\exp(\mathbf{w}^{(i)\top} \mathbf{h}_t^{(i)} + \beta^{(i)}) + 1 \right), \quad (26)$$

where $\mathbf{w}^{(i)} \in \mathbb{R}^R$ and $\beta^{(i)}$ are weight vector and bias, respectively.

Finally, the relational local random effect $b_t^{(i)}(\cdot)$ for node i at time t is sampled from a Gaussian distribution with zero mean and a variance given by σ^2 in Equation (26),

$$b_t^{(i)}(\cdot) \sim \mathcal{N}(0, \sigma_{i,t}^2). \quad (27)$$

The relational local random effect $b_t^{(i)}$ captures both past observations, covariates values of node i and its neighbors Γ_i for uncertainty estimates through $\sigma_{i,t}$, which is given by the probabilistic GCRN. A small $\sigma_{i,t}$ means a low uncertainty of prediction for node i at t . Specifically, the probabilistic model subsumes the point forecasting model when the relational local random effect is zero for all nodes at all timesteps as $\sigma_{i,t} = 0, \forall i \forall t$. The probabilistic property also allows the uncertainty to be propagated forward in time.

3.4.2 Estimating Uncertainties via Probabilistic DCRNN. We also describe a probabilistic DCRNN for the relational local component of GraphDF. For a given node i , its relational local random effect is derived with respect to its past observations, covariates and those of its neighbors, denoted by $\mathbf{Y}_t^{(i)} \in \mathbb{R}^{(|C|+1) \times P}$ as defined in Equation (25). The diffusion convolution models the relational local random effect among nodes. The GRU structure is adapted with the diffusion convolution to allow the random effects to be forwarded in time.

$$\mathbf{R}_t^{(i)} = \sigma \left(\Theta_R^{(i)} \star_{\mathcal{G}} [\mathbf{Y}_t^{(i)}, \mathbf{H}_t^{(i)}] + \mathbf{b}_R^{(i)} \right), \quad (28)$$

$$\mathbf{U}_t^{(i)} = \sigma \left(\Theta_U^{(i)} \star_{\mathcal{G}} [\mathbf{Y}_t^{(i)}, \mathbf{H}_t^{(i)}] + \mathbf{b}_U^{(i)} \right), \quad (29)$$

$$\mathbf{C}_t^{(i)} = \tanh \left(\Theta_C^{(i)} \star_{\mathcal{G}} [\mathbf{Y}_t^{(i)}, \mathbf{H}_t^{(i)}] + \mathbf{b}_C^{(i)} \right), \quad (30)$$

$$\mathbf{H}_t^{(i)} = \mathbf{U}_t^{(i)} \odot \mathbf{H}_{t-1}^{(i)} + (1 - \mathbf{U}_t^{(i)}) \odot \mathbf{C}_t^{(i)}, \quad (31)$$

where $\Theta_R^{(i)}, \Theta_U^{(i)}, \Theta_C^{(i)} \in \mathbb{R}^{P \times R}$ denote the parameters corresponding to different filters, $\mathbf{H}_t^{(i)} \in \mathbb{R}^{(|C|+1) \times R}$ is the hidden state for node i and its neighbors Γ_i , R is the number of hidden units

in the relational local model. $\mathbf{b}_I^{(i)}, \mathbf{b}_F^{(i)}, \mathbf{b}_C^{(i)}, \mathbf{b}_O^{(i)} \in \mathbb{R}^R$ are bias vector parameters. The graph convolution in equations above is performed with the submatrix $\mathbf{L}^{(i)}$ taken from the Laplacian matrix $\mathbf{L}$ of the graph G that explicitly models the important and meaningful dependencies between the multi-dimensional time-series data of each node. The matrix $\mathbf{L}^{(i)}$ consists of rows and columns corresponding to node i and its neighbors Γ_i . With the hidden state $\mathbf{H}_t^{(i)}$, the relational local random effect $b_t^{(i)}(\cdot)$ is calculated similarly to Equations (26) and (27).

ALGORITHM 1: Training Graph Deep Factor Models

- 1: Initialize the model parameters $\Phi (\mathbf{W}_*, \mathbf{b}_*, \Theta_*, \text{etc.})$
 - 2: **for** each time series pair $\{(\mathbf{z}^{(i)}, \mathbf{x}^{(i)})\}$ in a batch **do**
 - 3: With current model parameter estimates Φ
 - 4: Calculate the relational fixed effect $c_t^{(i)} = \sum_{k=1}^K w_{i,k} \cdot S_{i,k,t}$ described in Section 3.3
 - 5: Calculate the relational local random effect, $b_t^{(i)}(\cdot) \sim \mathcal{N}(0, \sigma_{i,t}^2)$ described in Section 3.4
 - 6: Compute marginal likelihood, $\mathbb{P}(\mathbf{z}^{(i)}) = \sum_t -\frac{1}{2} \ln(2\pi\sigma_{i,t}) - \sum_t \frac{(z_t^{(i)} - c_{i,t})^2}{2\sigma_{i,t}^2}$ and accumulate loss
 - 7: **end for**
 - 8: Compute the overall loss in the current batch and perform stochastic gradient descent and update the trainable parameters Φ accordingly.
-

3.5 Learning and Inference

To train a GraphDF model, we estimate the parameters Φ , which represent all trainable parameters ($\mathbf{W}$, etc.) in the relational global and relational local model, as well as the parameters in the embeddings. We leverage the maximum likelihood estimation,

$$\Phi = \operatorname{argmax}_{\Phi} \sum_i \mathbb{P}(\mathbf{z}^{(i)} | \Phi, \mathbf{A}, \{\mathbf{X}_{:,1:t}^{(j)}, \mathbf{z}_{1:t-1}^{(j)}\}_{j=1}^N), \quad (32)$$

where

$$\mathbb{P}(\mathbf{z}^{(i)}) = \sum_t -\frac{1}{2} \ln(2\pi\sigma_{i,t}) - \sum_t \frac{(z_t^{(i)} - c_{i,t})^2}{2\sigma_{i,t}^2} \quad (33)$$

is the negative log likelihood of Gaussian function. Notice that maximizing $-\frac{1}{2} \ln(2\pi\sigma_{i,t})$ will minimize the relational local random effect, at the same time, σ is small when the predicted fixed effect $c_{i,t}$ is close to the actual value $z_t^{(i)}$, as shown in the second term $\frac{(z_t^{(i)} - c_{i,t})^2}{2\sigma_{i,t}^2}$ in Equation (33). We describe a general training procedure in Algorithm 1.

3.6 Model Variants

In this section, we define a few of the GraphDF model variants investigated in Section 5.

- **GraphDF-GG:** This is the default model in our GraphDF framework, where we use a graph model to learn the K relational global factors (Section 3.3) and the probabilistic local graph component from Section 3.4.1 as the relational local model.
- **GraphDF-GR:** This model variant from the GraphDF framework uses the GCRN from Section 3.3 to learn the K relational global factors from the graph-based time-series data and leverages a simple RNN for modeling the local random effects of each node.

ALGORITHM 2: Incremental Online Learning for GraphDF

-
- 1: Initialize the model parameters Φ ($\mathbf{W}_*$, $\mathbf{b}_*$, Θ_* , etc.)
 - 2: **while** new time-series values $\{(z_t^{(i)}, x_t^{(i)})\}_{i=1}^N$ arrive at time t in stream **do**
 - 3: Add new values at time t and remove earliest for all N node time-series
 - 4: Perform incremental update to relational global model
 - 5: **for** each time series pair $(z^{(i)}, x^{(i)})$ **do**
 - 6: With current model parameter estimates Φ
 - 7: Calculate the relational fixed effect $c_t^{(i)} = \sum_{k=1}^K w_{i,k} \cdot S_{i,k,t}$ described in Section 3.3
 - 8: Calculate the relational local random effect $b_t^{(i)}(\cdot) \sim \mathcal{N}(0, \sigma_{i,t}^2)$ described in Section 3.4
 - 9: Compute marginal likelihood $\mathbb{P}(z^{(i)}) = \sum_t -\frac{1}{2} \ln(2\pi\sigma_{i,t}) - \sum_t \frac{(z_t^{(i)} - c_{i,t})^2}{2\sigma_{i,t}^2}$ and accumulate the loss
 - 10: **end for**
 - 11: **end while**
 - 12: Compute the overall loss in the current batch and perform SGD and update the trainable parameters Φ accordingly.
-

- **GraphDF-RG:** This model variant from the GraphDF framework uses a simple RNN to learn the K global factors and fixed effects of the nodes and for the relational random effects of the nodes, we leverage the probabilistic graph component from Section 3.4.1 as the relational local model.

The GraphDF framework is flexible with many interchangeable components. Importantly, the relational global component (Section 3.3) of GraphDF is completely interchangeable. In particular, this component uses the graph-based time-series data to learn the K global factors and fixed effects of the nodes. Similarly, one can also leverage any arbitrary relational local model (Section 3.4) for obtaining the relational local random effects of the nodes.

4 INCREMENTAL ONLINE LEARNING FOR GRAPHDF

In practical setting, time-series are changing frequently in a streaming fashion as new time-series observations arrive for all N nodes. For instance, in Google cloud dataset [61] that we used, the time interval is five minutes, which means we have new time-series observations for all N nodes every five minutes. In such scenarios, we want to incrementally update the forecasting model without the need to relearn the entire model from scratch every time a new point arrives in the stream.

However, the original GraphDF model is incapable of incrementally updating the model as new values arrive in the stream. One solution could be to retrain a new GraphDF model from scratch each time new values arrive; however, it will take too much time for GraphDF to initialize new parameters and train from scratch, which would cause a waste of limited computing resources, especially when predicting with large-scale time-series data. To handle this, we propose an incremental online approach called IOGraphDF that efficiently updates the current model, without the need to retrain it entirely from scratch with each newly arrive point. By doing this, the IOGraphDF modeling operates much more efficiently.

Denote t the moment dynamic streaming time-series currently reach, we define the set of covariate time-series as $\mathcal{X}_t = \{\mathbf{X}^{(i)}\}_{i=1}^N$ where $\mathbf{X}^{(i)} \in \mathbb{R}^{D \times t}$, where D is the number of dimension for covariate features. We define the set of target time-series is $\mathcal{Z}_t = \{z^{(i)}\}_{i=1}^N$ where $z_{1:t}^{(i)}$ denotes the i th univariate target time-series. Given $(\mathcal{X}_t, \mathcal{Z}_t)$ at the moment t , our task is to give probabilistic

predictions at the horizon for each target time-series $\{\hat{\mathbf{z}}_{t+1}^{(i)}\}_{i=1}^N, \{\hat{\mathbf{z}}_{t+2}^{(i)}\}_{i=1}^N, \dots, \{\hat{\mathbf{z}}_{t+\dots}^{(i)}\}_{i=1}^N$. Hence the target function is modified accordingly as

$$\mathbb{P}\left(\left\{\mathbf{z}_{t+1:t+\tau}^{(i)}\right\}_{i=1}^N \mid \mathbf{A}, \left\{\mathbf{z}_{1:t}^{(i)}, \mathbf{X}_{:,1:t+\tau}^{(i)}\right\}_{i=1}^N; \Phi\right) \quad t = 1, 2, \dots \quad (34)$$

Therefore, every time new observations arrive, the new problem formulation is conditioned upon all available known values to make further predictions. The algorithm for IOGraphDF is described in Algorithm 2.

Now we analyze the time complexity of IOGraphDF, when a single value arrives at time t , the worst-case time complexity is

$$O((|E| \cdot KLkN + K) + LC_{\max}kN), \quad (35)$$

where $|E|$ is size of the edge set in the graph, K is the number of relational global factors, L is the order of the graph convolution. Noticeably, for IOGraphDF, we maintain only the most recent k values in a streaming window for each of N nodes. The time complexity of the relational global component can be decomposed into the computation for K relational global factors, and their linear combination (*i.e.*, the K term in Equation (35)) which is timewise negligible, therefore, the time complexity of the relational global component $O(|E| \cdot KLkN + K)$ is approximately linear to all the aforementioned values.

Further in the time complexity $O(LC_{\max}kN)$ for the relational local component, $C_{\max}$ is the maximum node degree of the graph, and thus Equation (35) is the worst case. However, in practice $NC_{\max} \gg \sum_{i=1}^N |C_i|$, where $|C_i|$ is the degree of the i th node. Notice that since the number of local iterations in the online model is a small fixed constant (*e.g.*, 1, 10) for every new data point that arrives at time t , it can safely be omitted. In the multi-step ahead prediction scenario, Equation (35) is multiplied by a factor of future horizon τ , *i.e.*, the time complexity is linear to τ . However, the offline model time complexity is significantly larger by a factor of epoch numbers M . In the offline case of GraphDF, we are given T time-series values for each N nodes, and need to relearn an entire model from scratch every time. Thus, the time complexity of GraphDF is significantly higher than that of IOGraphDF, $O(M \cdot (|E| \cdot KLTN + K) + LC_{\max}TN)) \gg O((|E| \cdot KLkN + K) + LC_{\max}kN)$. It is important to note that T increases as a function of the stream size, hence, the offline model consumes much more computation time than the IOGraphDF model.

In terms of input data space requirements, the offline GraphDF requires $O(TN)$ space while the IOGraphDF requires only $O(kN)$ space where w is the most recent w value in the stream. Hence, since $k \ll T$, then $O(kN) \ll O(TN)$. Furthermore, as more data arrives over time, we can also see that the input data space of GraphDF can actually increase (assuming that it is trained using all available data). This is in contrast to the IOGraphDF that always uses a fixed amount of space, as when a new data point arrives for time t , we simply discard the most distant value and append the new value.

5 EXPERIMENTS

In this section, We examine the performance of GraphDF models with previous state-of-the-arts, then we evaluate the performance of IOGraphDF models against GraphDF models, finally, we investigate both models in the task of opportunistic scheduling. For GraphDF, the experiments are designed to investigate the following:

- RQ1. Does GraphDF outperform the state-of-the-art deep probabilistic forecasting method?
- RQ2. Are the GraphDF models fast and scalable for large-scale time-series forecasting?
- RQ3. Can GraphDF generate cloud usage forecasts to effectively perform opportunistic workload scheduling?

Table 1. Statistics of the Two Real-world Large-scale Collections of Time-series

Data	V	E	Density	Avg. Deg.	Median Deg.	Mean wDeg.	D	Time-scale	T	Mean CPU usage	Median CPU usage
Google	12,580	1,196,658	0.0075	95.1	40	30.3	5	5 min	8,354	22.7%	21.4%
Adobe	3,270	221,984	0.0207	67.9	15	67.7	5	30 min	1,687	108.5%	9.1%

5.1 Experimental Setup

We used two real-world datasets in our experiments; Google trace data and Adobe trace data. Table 1 shows the statistics and properties (e.g., edge density and average degree).

Google Trace. The Google trace dataset records the activities of a cluster of 12,580 machines for 29 days since 19:00 EDT on May 1, 2011. The CPU and memory usage for each task are recorded every 5 minutes. The usage of tasks is aggregated to the usage of associated machines, resulting time-series of length 8,354.

Adobe Workload Trace. The Adobe trace dataset records the CPU and memory usage of 3,270 nodes in the period from October 31 to December 5 in 2018. The timescale is 30 minutes, resulting time-series of length 1,687.

For the opportunistic workload scheduling case study in Section 6, we need to train a model *fast* within a few minutes and then forecast a single as well as multiple timesteps ahead, which are then used to make a decision on whether the current resources are enough or if we should instead scale up or down. Therefore, models must be able to be trained fast within a few minutes. To ensure the models are trained fast within minutes, we use six observations in the time-series data for training across all experiments. Furthermore, as in most time-series forecasting problems, the future CPU usage of machines is highly dependent on the most recent observations than those in the distant past. We set the number of embedding dimensions as $K = 10$ in $\mathbf{w}_i \in \mathbb{R}^K$ and use time feature series as covariates. We set the embedding dimension to $K = 10$ in $\mathbf{w}_i \in \mathbb{R}^K$ and used $D = 5$ covariates for each time-series. Similar to DF [75], the time features (e.g., minute of hour, hour of day) are used as covariates. We derive a fixed graph using RBF on the past observations.

The three models described in Section 3.6 are evaluated against four state-of-the-art probabilistic forecasting methods including Deep Factors, DeepAR [31], MQRNN [76], and NBEATS [57]:

- *Deep Factors* is a generative approach that combines a global model and a local model. To ensure a fair comparison, we modified DF to solve the same problem formulated in Equation (1), and thus the DF version used for comparison uses the same inputs as GraphDF. Unless otherwise mentioned, we use the same experimental setup as mentioned in the DF article. In particular, as suggested by the authors, we use the Gaussian likelihood in terms of the random effects in the deep factors model. We use 10 global factors with an LSTM cell of 1-layer and 50 hidden units in its global component, and 1-layer and five hidden units RNN in the local component. We also use suggested hyperparameters for other compared baselines.
- *DeepAR* is an RNN-based global model, we use an LSTM layer with 50 hidden units in DeepAR.
- *MQRNN* is a sequence model with quantile regression and NBEATS is an interpretable pure deep learning model. For MQRNN, we use a GRU bidirectional layer with 50 hidden units as encoder and a modified forking layer in the decoder.
- For *N-BEATS*, we use an ensemble modification of the model and take the median value from 10 bagging bases as results.

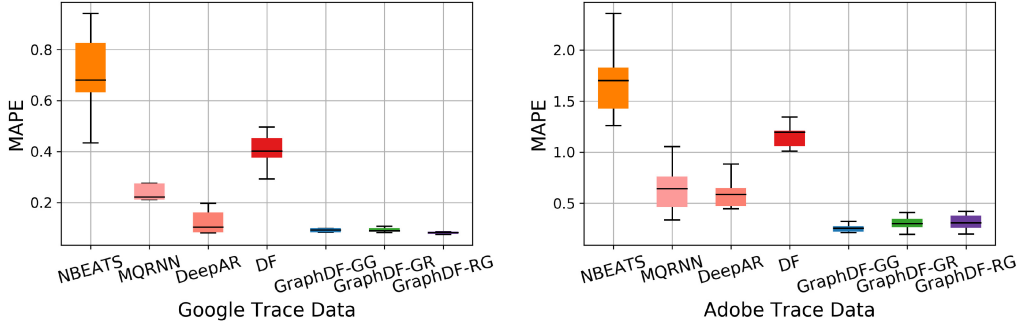


Fig. 3. Probabilistic forecasting results for 3-step ahead forecast horizon (P50QL) from Adobe and Google trace dataset.

All methods are implemented using MXNet Gluon [2, 16]. The Adam optimization method is used with a default initial learning rate of 0.001 to train all models. The training epochs are selected by grid search in $\{100, 200, \dots, 1000\}$. An early stopping strategy is leveraged if weight losses do not decrease for 10 continuous epochs. We used a learning rate decay factor of 0.5, minimum learning rate of $5 * 10^{-5}$, Xavier as the weight initializer, and trained for 500 epochs on Adobe data and 100 epochs for the Google dataset. Some hyperparameters are specific to our method: In GraphDF, we set the order $L = 2$ in Equation (10). A small number of the order indicates that the model makes forecasts based more on neighboring nodes than those more distant. For other methods, we use default hyperparameters given by the Gluons implementation if not otherwise mentioned.

To evaluate the probabilistic forecasts, we use the quantile loss defined as follows: given a quantile $\rho \in (0, 1)$, a target value z_t and ρ -quantile prediction $\hat{z}_t(\rho)$, the ρ -quantile loss is

$$QL_{\rho}[z_t, \hat{z}_t(\rho)] = 2 \left[\rho(z_t - \hat{z}_t(\rho)) \mathbb{I}_{z_t - \hat{z}_t(\rho) > 0} + (1 - \rho)(\hat{z}_t(\rho) - z_t) \mathbb{I}_{z_t - \hat{z}_t(\rho) \leq 0} \right]. \quad (36)$$

For deriving quantile losses over a timespan across all time-series, we use a normalized version of quantile loss $\sum_{i,t} QL_{\rho}[z_{i,t}, \hat{z}_{i,t}(\rho)] / \sum_{i,t} |z_{i,t}|$. When $\rho = 0.5$, the resulted quantile loss is equivalent to **Mean Absolute Percentage Error (MAPE)**. In experiments, the quantile losses are computed based on 100 sample values. Our algorithms are implemented in MXNet Gluon [16] and all experiments run on a machine with 8 CPU cores.

5.2 Forecasting Performance

To answer the research questions, we investigate the proposed GraphDF framework with various forecast horizons including $\tau = \{1, 3, 4, 5\}$.

The results for single and multi-step ahead forecasting are provided in Table 2 and Table 3–5, respectively, where the best result for every dataset and forecast horizon are highlighted in bold. We run 10 trials for each model and report the average for $\rho = \{0.1, 0.5, 0.9\}$, denoted as the P10QL, P50QL, and P90QL, respectively. In all cases, we observe that the GraphDF models outperform the state-of-the-art method across all datasets and forecast horizons. Furthermore, we observe that in most cases, the GraphDF-GG variant that uses GCRN for both the relational global and relational local components outperforms the other variants. The second best GraphDF model is GraphDF-GR followed by GraphDF-RG.

To understand the overall performance and variance of the models, we show boxplots for each model in Figure 3. Strikingly, we observe that the GraphDF models provide more accurate forecasts with significantly lower variance.

Table 2. Results for One-step ahead Forecasting (p10QL, p50QL and p90QL)

	DATA	NBEATS	MQRNN	DeepAR	DF	GraphDF-GG	GraphDF-GR	GraphDF-RG
p10QL	Google	18.12 $\pm$ 201.26	0.190 $\pm$ 0.004	0.046 $\pm$ 0.000	0.083 $\pm$ 0.001	0.037 $\pm$ 0.000	0.038 $\pm$ 0.000	0.044 $\pm$ 0.000
	Adobe	0.615 $\pm$ 0.091	0.132 $\pm$ 0.000	0.164 $\pm$ 0.001	1.128 $\pm$ 0.004	0.118 $\pm$ 0.001	0.119 $\pm$ 0.000	1.027 $\pm$ 2.700
p50QL	Google	10.064 $\pm$ 62.12	0.172 $\pm$ 0.001	0.098 $\pm$ 0.001	0.239 $\pm$ 0.001	0.072 $\pm$ 0.000	0.076 $\pm$ 0.000	0.077 $\pm$ 0.000
	Adobe	3.070 $\pm$ 2.286	0.272 $\pm$ 0.001	0.619 $\pm$ 0.026	1.649 $\pm$ 0.001	0.188 $\pm$ 0.000	0.210 $\pm$ 0.001	0.746 $\pm$ 0.835
p90QL	Google	2.013 $\pm$ 2.485	0.106 $\pm$ 0.001	0.051 $\pm$ 0.000	0.144 $\pm$ 0.002	0.041 $\pm$ 0.000	0.044 $\pm$ 0.000	0.048 $\pm$ 0.000
	Adobe	5.524 $\pm$ 7.410	0.217 $\pm$ 0.000	0.949 $\pm$ 0.086	1.802 $\pm$ 0.002	0.153 $\pm$ 0.001	0.169 $\pm$ 0.001	0.342 $\pm$ 0.037

Table 3. Results for Multi-step ahead Forecasting (p10QL)

DATA	H	NBEATS	MQRNN	DeepAR	DF	GraphDF-GG	GraphDF-GR	GraphDF-RG
Google	3	0.652 $\pm$ 0.396	0.152 $\pm$ 0.006	0.070 $\pm$ 0.000	0.132 $\pm$ 0.004	0.064 $\pm$ 0.001	0.077 $\pm$ 0.000	0.087 $\pm$ 0.000
	4	0.260 $\pm$ 0.017	0.272 $\pm$ 0.018	0.138 $\pm$ 0.000	0.193 $\pm$ 0.016	0.071 $\pm$ 0.001	0.083 $\pm$ 0.000	0.089 $\pm$ 0.001
	5	0.447 $\pm$ 0.054	0.147 $\pm$ 0.005	0.484 $\pm$ 0.017	0.327 $\pm$ 0.036	0.054 $\pm$ 0.000	0.113 $\pm$ 0.001	0.088 $\pm$ 0.001
Adobe	3	0.811 $\pm$ 0.295	0.184 $\pm$ 0.003	0.207 $\pm$ 0.002	0.303 $\pm$ 0.006	0.183 $\pm$ 0.002	0.216 $\pm$ 0.008	0.267 $\pm$ 0.009
	4	0.985 $\pm$ 0.537	0.219 $\pm$ 0.008	0.273 $\pm$ 0.003	0.313 $\pm$ 0.018	0.184 $\pm$ 0.002	0.242 $\pm$ 0.014	0.423 $\pm$ 0.019
	5	0.626 $\pm$ 0.023	0.398 $\pm$ 0.229	0.402 $\pm$ 0.011	0.343 $\pm$ 0.047	0.251 $\pm$ 0.016	0.298 $\pm$ 0.031	0.544 $\pm$ 0.020

Table 4. Results for Multi-step ahead Forecasting (p50QL)

DATA	H	NBEATS	MQRNN	DeepAR	DF	GraphDF-GG	GraphDF-GR	GraphDF-RG
Google	3	0.741 $\pm$ 0.050	0.257 $\pm$ 0.011	0.148 $\pm$ 0.001	0.400 $\pm$ 0.004	0.091 $\pm$ 0.001	0.134 $\pm$ 0.002	0.097 $\pm$ 0.000
	4	0.618 $\pm$ 0.105	0.410 $\pm$ 0.017	0.191 $\pm$ 0.000	0.454 $\pm$ 0.007	0.097 $\pm$ 0.002	0.185 $\pm$ 0.002	0.109 $\pm$ 0.000
	5	0.485 $\pm$ 0.021	0.684 $\pm$ 0.012	0.466 $\pm$ 0.006	0.563 $\pm$ 0.017	0.128 $\pm$ 0.000	0.284 $\pm$ 0.012	0.126 $\pm$ 0.001
Adobe	3	1.683 $\pm$ 0.100	0.556 $\pm$ 0.028	0.592 $\pm$ 0.017	1.116 $\pm$ 0.006	0.272 $\pm$ 0.004	0.315 $\pm$ 0.006	0.319 $\pm$ 0.005
	4	1.424 $\pm$ 0.210	0.574 $\pm$ 0.011	0.629 $\pm$ 0.024	1.029 $\pm$ 0.001	0.314 $\pm$ 0.004	0.353 $\pm$ 0.007	0.405 $\pm$ 0.007
	5	1.069 $\pm$ 0.027	0.687 $\pm$ 0.064	0.633 $\pm$ 0.012	1.039 $\pm$ 0.004	0.375 $\pm$ 0.007	0.401 $\pm$ 0.014	0.484 $\pm$ 0.005

5.3 Runtime Analysis

Training and inference runtime performance results are shown in Tables 6 and 7, respectively. All forecasting models are trained using only six previous values for each time-series in the collection (Table 6). As expected, the relational global model is significantly faster and more scalable than the global model used in Deep Factors. In particular, we see that the runtime of our GraphDF model that uses GCRN for the relational global model with RNN as the local model is significantly faster than Deep Factors that uses the same local model, but differs in the global model used. This is due to the fact that in the state-of-the-art DF model, all time-series are considered equivalently and jointly when learning the K global factors. This can be thought of as a fully connected graph where each time-series is connected to every other time-series. In comparison, the relational global components of GraphDF leverage the graph that encodes explicit dependencies between the different time-series, and therefore, does not need to leverage all pairwise time-series, but only a smaller fraction of those that are actually related.

In terms of inference, all models are fast taking only a few seconds as shown in Table 7. For inference, we report the time (in seconds) to infer the next six values in each time-series in the collection. In all cases, the GraphDF model variants are significantly faster than DF across both Google and Adobe workload datasets.

Table 5. Results for Multi-step ahead Forecasting (p90QL)

DATA	H	NBEATS	MQRNN	DeepAR	DF	GraphDF-GG	GraphDF-GR	GraphDF-RG
Google	3	0.830 ± 0.262	0.091 ± 0.001	0.067 ± 0.000	0.208 ± 0.002	0.051 ± 0.000	0.051 ± 0.000	0.089 ± 0.000
	4	0.976 ± 0.429	0.090 ± 0.000	0.070 ± 0.000	0.213 ± 0.002	0.050 ± 0.001	0.076 ± 0.001	0.095 ± 0.001
	5	0.523 ± 0.085	0.124 ± 0.000	0.134 ± 0.000	0.220 ± 0.002	0.069 ± 0.001	0.167 ± 0.013	0.094 ± 0.001
Adobe	3	2.556 ± 0.328	0.317 ± 0.002	0.751 ± 0.117	1.545 ± 0.008	0.248 ± 0.004	0.254 ± 0.003	0.301 ± 0.003
	4	1.862 ± 1.212	0.335 ± 0.004	0.696 ± 0.170	1.673 ± 0.008	0.317 ± 0.006	0.318 ± 0.009	0.482 ± 0.014
	5	1.512 ± 0.082	0.463 ± 0.003	0.546 ± 0.000	1.690 ± 0.015	0.410 ± 0.021	0.434 ± 0.007	0.512 ± 0.007

Table 6. Training Runtime Performance (in Seconds)

DATA	NBEATS	MQRNN	DeepAR	DF	GraphDF-GG	GraphDF-GR	GraphDF-RG
Google	663.31 ± 54.09	284.76 ± 71.08	413.79 ± 49.62	315.06 ± 67.80	279.45 ± 41.19	222.08 ± 69.52	281.76 ± 49.51
Adobe	462.06 ± 120.07	393.08 ± 4.22	351.99 ± 285.30	378.97 ± 441.64	282.30 ± 36.80	211.20 ± 21.56	264.00 ± 56.29

Table 7. Inference Runtime Performance (in Seconds)

DATA	NBEATS	MQRNN	DeepAR	DF	GraphDF-GG	GraphDF-GR	GraphDF-RG
Google	88.08 ± 10.96	9.22 ± 0.06	17.06 ± 0.16	8.28 ± 0.02	1.67 ± 0.03	0.99 ± 0.003	1.16 ± 0.003
Adobe	162.63 ± 7.59	2.69 ± 0.006	4.30 ± 0.02	2.12 ± 0.001	0.51 ± 0.005	0.28 ± 0.001	0.33 ± 0.000

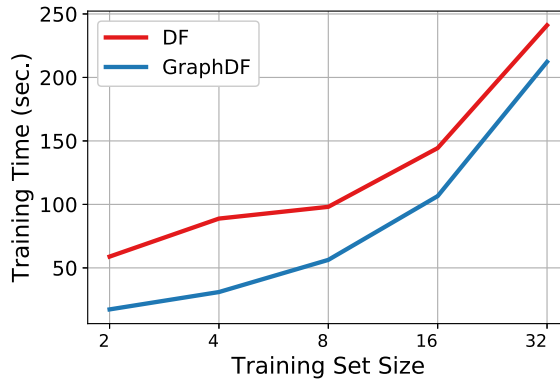


Fig. 4. Comparing scalability of GraphDF to DF as the training set size increases for the Adobe workload trace dataset. Note training set size = data points per time-series.

5.4 Scalability

To evaluate the scalability of GraphDF, we vary the training set size (*i.e.*, the number of previous data points per time-series to use) from $\{2, 4, 8, 16, 32\}$. Figure 4 shows that GraphDF scales nearly linear as the training set size increases from 2 to 32. For instance, GraphDF takes around 15 seconds to train using only two data points per time-series and 30 seconds using four data points per time-series, and so on. We also observe that for the Adobe data, GraphDF is always about 3 times faster compared to DF across all training set sizes.

5.5 Experimental Result on IOGraphDF

To evaluate the performance of IOGraphDF, we design experiments to answer the following research questions:

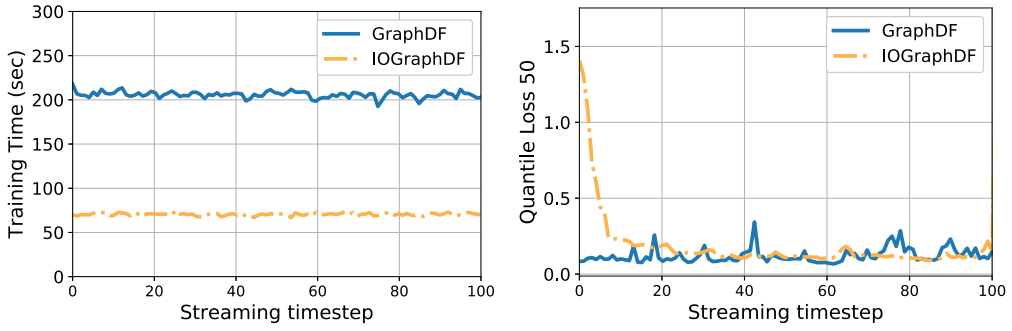


Fig. 5. Comparing training runtime and one step ahead P50QL (*i.e.*, MAPE) between GraphDF (blue) and IOGraphDF (yellow dashed line) on Google dataset.

RQ4. Does IOGraphDF yield more accurate predictions than GraphDF?

RQ5. Does IOGraphDF outperform GraphDF regarding training and prediction runtime?

RQ6. Does IOGraphDF perform better in the opportunistic workload scheduling task?

Experimental Setting. To compare the performance and runtime between IOGraphDF and GraphDF model, we simulate a streaming procedure containing 100 timesteps, where new values are received by both models at each time step. At each time step, new values arrive, a new GraphDF instance is initialized and then trained with the just arrived values. By contrast, IOGraphDF model only needs to be initialized once at the beginning, and the same IOGraphDF instance is incrementally updated from the previous time step with newly arrived data. Incoming values are assumed to be more dependent on near observations than those in the distant past, therefore, we maintain a shifting window size of 9 to include the most recent observations relative to t . When t increments, the oldest values are removed from the window and newly arrived values are appended to the window. The window values are split sets for data training and fitting procedure. Using the Google dataset, for GraphDF model, the number of training epochs is set as 100, while for IOGraphDF model, the number of local iterations (the training times upon each shifting window) only needs to be set as a smaller number 50, because after the model is initialized, the existed model preserve learned information from previous time steps. In the inference stage, values in $\tau = 3$ steps ahead are predicted and evaluated with ground truth.

Experimental Results. Overall, we observe from Figure 5 (left) and Table 8 that IOGraphDF is significantly faster, with very comparable loss (as shown in Figure 5 (right) and Table 8). Hence, IOGraphDF sacrifices a small amount of accuracy for a significant speedup.

We further conducted experiments on 100 timesteps using the best variants GraphDF-GG and IOGraphDF-GG. As observed in Figure 5 (Right), the p50QL (*i.e.*, MAPE) of GraphDF (in blue) and IOGraphDF (yellow dashed line) for each 100 streaming timestep is plotted. In the first few timesteps, IOGraphDF has much higher errors than GraphDF; however, IOGraphDF performance converges quickly to that of GraphDF's, as two lines in Figure 5 are mostly overlapping after streaming timestep 20.

In both offline and online models, the expected runtime on training is positively related to the number of values on input and output. In the offline GraphDF model, each time new values arrived, a new model has to be created and trained using the new values, and only recent values are taken as input, hence, this causes the loss of earlier observations as possibly useful information. In our proposed IOGraphDF, the input is only modeled upon a fixed amount of recent values. Since the same model instance is used and kept updated over time, IOGraphDF

Table 8. Results for Multi-step ahead Forecasting (RUNTIME) with IOGraphDF

DATA	H	GraphDF-GG	GraphDF-GR	GraphDF-RG	IOGraphDF-GG	IOGraphDF-GR	IOGraphDF-RG
Google	3	279.45 $\pm$ 41.19	222.08 $\pm$ 69.51	281.76 $\pm$ 49.51	70.209 $\pm$ 0.844	51.213 $\pm$ 1.380	68.237 $\pm$ 1.477
	4	282.67 $\pm$ 12.33	222.31 $\pm$ 14.31	283.54 $\pm$ 26.26	70.325 $\pm$ 1.227	51.481 $\pm$ 1.222	68.340 $\pm$ 0.729
	5	283.69 $\pm$ 15.43	223.53 $\pm$ 24.54	289.16 $\pm$ 32.79	70.839 $\pm$ 1.351	51.733 $\pm$ 1.465	68.807 $\pm$ 1.080
Adobe	3	282.30 $\pm$ 36.80	211.20 $\pm$ 21.36	264.00 $\pm$ 56.29	27.69 $\pm$ 1.17	20.20 $\pm$ 0.82	25.19 $\pm$ 0.55
	4	282.80 $\pm$ 26.03	212.60 $\pm$ 14.00	264.90 $\pm$ 60.90	26.69 $\pm$ 0.63	20.49 $\pm$ 1.17	25.22 $\pm$ 0.49
	5	287.30 $\pm$ 12.03	214.81 $\pm$ 21.93	274.78 $\pm$ 46.10	26.86 $\pm$ 0.82	20.78 $\pm$ 0.73	25.24 $\pm$ 0.68

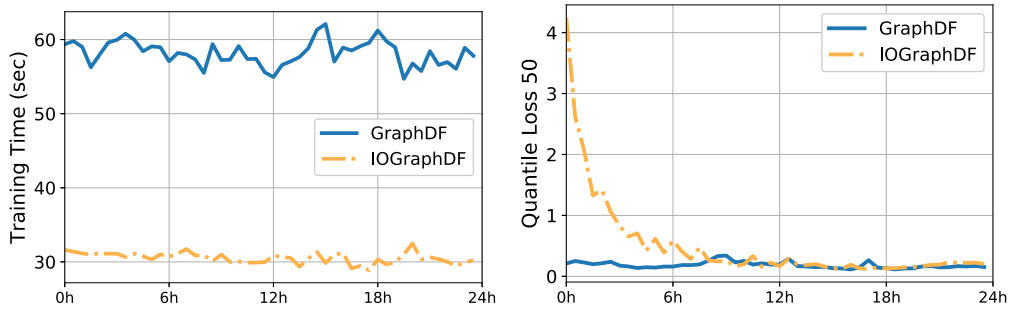


Fig. 6. Comparing training runtime and one step ahead P50QL between GraphDF (blue) and IOGraphDF (yellow dashed line) on Adobe dataset.

has the advantage of leveraging earlier observations for forecasting, as learned by the model parameters. As a consequence, we set the local iterations, the number of times data values are used to update IOGraphDF model, a small number 50, which results a much faster IOGraphDF while still preserving a high accuracy. The runtime comparison between GraphDF and IOGraphDF over 100 timesteps is shown in Figure 5 (Left). A similar result can be observed from the experiment using Adobe dataset using the same warm start period (20 steps *i.e.*, 10 hours), as shown in Figure 6. We also summarize the average and deviation runtime in Table 9.

Following the setup in Section 5.2, we design experiments to investigate the performance of IOGraphDF model variants over multi-step ahead prediction, and compare them against previous results from GraphDF variants. Since IOGraphDF models take advantages of incremental training, for a fair comparison, we report the result for IOGraphDF models after a *warm start* period (set as 20 timesteps). The values in the warm start period are used to train the IOGraphDF models, but not used for loss comparison. The values after the warm period are then used to evaluate and compare the prediction loss with the GraphDF variants. The result is shown in Table 10. We observe that on the Google dataset, IOGraphDF variants reach very close result to the GraphDF counterparts, while IOGraphDF models only require significantly less training runtime than GraphDF models, as shown in Table 8. For the Adobe dataset, IOGraphDF models not only require less runtime but also obtain more accurate predictions with smaller losses in most cases (highlighted in the column IOGraphDF-GG).

5.6 Ablation Study

We further investigate the effects of hyperparameters upon the IOGraphDF model with extensive experiments. The length of the warm start period and the number of relational global factors K are selected from the range of $\{5, 10, 20, 50\}$. We report p50QL for forecasting 3-step ahead on Google dataset with combinations of these hyperparameters and the result is shown in Table 11. From the

Table 9. Runtime Comparison between GraphDF and IOGraphDF over Timesteps

Data	GraphDF	IOGraphDF
Google	208.770 $\pm$ 11.862	76.948 $\pm$ 22.717
Adobe	58.170 $\pm$ 1.666	30.534 $\pm$ 0.718

Table 10. Results for Multi-step ahead Forecasting (p50QL) with IOGraphDF

DATA	H	GraphDF-GG	GraphDF-GR	GraphDF-RG	IOGraphDF-GG	IOGraphDF-GR	IOGraphDF-RG
Google	3	0.091 $\pm$ 0.001	0.134 $\pm$ 0.002	0.097 $\pm$ 0.000	0.104 $\pm$ 0.006	0.146 $\pm$ 0.007	0.141 $\pm$ 0.013
	4	0.097 $\pm$ 0.002	0.185 $\pm$ 0.002	0.109 $\pm$ 0.000	0.108 $\pm$ 0.005	0.146 $\pm$ 0.005	0.156 $\pm$ 0.005
	5	0.128 $\pm$ 0.000	0.284 $\pm$ 0.012	0.126 $\pm$ 0.001	0.122 $\pm$ 0.006	0.164 $\pm$ 0.010	0.171 $\pm$ 0.010
Adobe	3	0.272 $\pm$ 0.004	0.315 $\pm$ 0.006	0.319 $\pm$ 0.005	0.211 $\pm$ 0.023	0.328 $\pm$ 0.039	0.371 $\pm$ 0.042
	4	0.314 $\pm$ 0.004	0.353 $\pm$ 0.007	0.405 $\pm$ 0.007	0.240 $\pm$ 0.018	0.332 $\pm$ 0.057	0.393 $\pm$ 0.056
	5	0.375 $\pm$ 0.007	0.401 $\pm$ 0.014	0.484 $\pm$ 0.005	0.286 $\pm$ 0.023	0.341 $\pm$ 0.083	0.404 $\pm$ 0.059

Table 11. Results on Combination of Hyperparameters for 3-step ahead Forecasting (p50QL) with Google Dataset, the Best Performance for each Number of Warm Start Period (row) is Highlighted in Bold

		Number of relational global factors K			
		5	10	20	50
Number of warm start period	5	1.121 $\pm$ 0.124	0.804 $\pm$ 0.105	0.746 $\pm$ 0.058	0.453 $\pm$ 0.048
	10	0.321 $\pm$ 0.015	0.305 $\pm$ 0.011	0.295 $\pm$ 0.010	0.281 $\pm$ 0.009
	20	0.124 $\pm$ 0.012	0.104 $\pm$ 0.006	0.108 $\pm$ 0.009	0.126 $\pm$ 0.008
	50	0.102 $\pm$ 0.005	0.009 $\pm$ 0.003	0.105 $\pm$ 0.007	0.107 $\pm$ 0.003

result table, we observe (1) When the number of relational global factor K is fixed, i.e., for each column of Table 11, the longer the warm start period is, the better performance IOGraphDF achieves. (2) When the number of warm start period is small, the performance is drastically improved as the number of relational global factors K increases, as shown in the first two rows of Table 11, and the best performance is achieved when $K = 50$. We suggest this can be due to the insufficient training of the model since the number of warm start period is small. (3) When the number of warm start period is large, the performance improves little or even worsens as the number of relational global factors K increases, as shown in the last two rows of Table 11. This can be caused due to the excessive amount of model parameters which leads to overfitting.

6 CASE STUDY: OPPORTUNISTIC SCHEDULING

We leverage our GraphDF forecasting model to enable the opportunistic scheduling of batch workloads. Since batch workloads such as ML training, crawling web pages and so on have loose latency requirements, they can be scheduled on underutilized resources (such as CPU cores) opportunistically. This improves resource utilization of the cluster and reduces operating costs by precluding the need to allocate additional machines to run the batch workloads. Our model generates probabilistic CPU usage forecasts for cloud nodes and nodes with low predicted usage are employed for scheduling these workloads.

Our model satisfies the following requirements of such an opportunistic scheduler. First, the model must be able to correctly forecast utilization. If the utilization is underestimated, tasks will be assigned to busy machines and then need to be canceled, which is a waste of resources. Second,

ALGORITHM 3: Opportunistic Scheduling

```

1: Initialize hyperparameters and variables lookback window for GraphDF (or streaming win-
   window for IOGraphDF)  $w$ , horizon  $\tau$ , threshold ratio  $\epsilon$ , portion ratio  $\lambda$ . Accumulated utilization
   improvement  $Acc$ 
2: while New observations arrive  $t \leftarrow t + 1$  do
3:   Initialize weights  $\Phi$  for new model  $f_t$ 
4:    $f_t \leftarrow \mathbb{P}(\cdot | \Phi, \mathbf{A}, \{\mathbf{X}_{t-w+1:t+\tau}^{(i)}, \mathbf{Z}_{t-w+1:t}^{(i)}\}_{i=1}^N)$ 
5:   for each node  $i$  do
6:     Obtain forecasts  $\hat{z}^{(i)} = \{\hat{z}_{t+1}^{(i)}, \hat{z}_{t+2}^{(i)} \dots \hat{z}_{t+\tau}^{(i)}\} \sim f_t$ 
7:     if MEAN of forecasts  $\text{MEAN}(\hat{z}^{(i)}) \leq \epsilon$  then
8:       Utilization  $Acc \leftarrow Acc + \lambda(1 - \text{MEAN}(\hat{z}^{(i)}))$ 
9:     else
10:      Cancel assigned tasks on node  $i$ 
11:    end if
12:  end for
13: end while

```

the execution time of the forecasting model must be significantly faster than the time period used for data collection, *e.g.*, since CPU usage in Google dataset is observed every 5 minutes, the CPU usage forecast should be generated in less than 5 minutes *i.e.*, before the next observation arrives. We simulate opportunistic scheduling by developing two main components, the *forecaster* and the *scheduler*. The Google dataset provides CPU usage for the cluster in this study. The forecaster reads the six most recent observed CPU utilization values of each machine from the data stream and predicts the next three values. The scheduler identifies underutilized machines as those with mean predicted utilization across the three predictions lower than a predefined threshold ϵ (25%). To safely make use of the idle resources without disturbing already running tasks or cause thrashing, the scheduler only assigns workloads that require at most 75% of compute resources. If a machine is assigned batch workloads that exceed resource availability, they are *terminated/canceled*. This procedure is described in Algorithm 3.

Effects on CPU Utilization. Figure 7 shows CPU utilization without opportunistic workload scheduling (*vanilla*) and with scheduling based on each forecaster over a period of 6 hours on the Google dataset. We observe that the GraphDF-based forecaster consistently outperforms both *vanilla* and DF-based versions by generating forecasts with higher accuracy. We also observed similar results (in Figure 9) over longer periods (12 hours and 24 hours) for Google dataset. Table 12 summarizes the performance of the GraphDF-based forecaster with respect to three metrics *CPU utilization improvement*, *correct scheduling ratio*, and *cancellation ratio*. The utilization improvement measures the absolute increase in CPU usage compared to the *vanilla* version. Correct scheduling ratio corresponds to the ratio when the predicted utilization by the scheduler matches the actual utilization. Cancellation ratio measures the fraction of machines on which the batch workload was terminated due to incorrectly generated forecasts. We observe that GraphDF-based workload scheduling leads to higher CPU utilization, higher correct scheduling ratio, and lower cancellation ratio compared to DF-based scheduling.

Execution Time Comparison. Figure 8 shows that the runtime of DF-based scheduling often exceeds the 5-minute time limit, while the GraphDF-based version is much faster and always meets it. The average time of prediction by DF is 340.43 ± 77.95 where the average time of prediction by GraphDF is 231.90 ± 4.25 . Hence, GraphDF persuasively provides a solution for enhancing

Table 12. Results for Opportunistic Scheduling in the Cloud over a 6 Hour Period using Different Forecasting Models

Data	Model	utilization improvement (%)	correct ratio (%)	cancellation ratio (%)
Google	DF	38.8	68.6	20.9
	GraphDF	41.9	88.6	8.2
Adobe	DF	42.0	65.8	19.1
	GraphDF	53.2	97.0	2.2

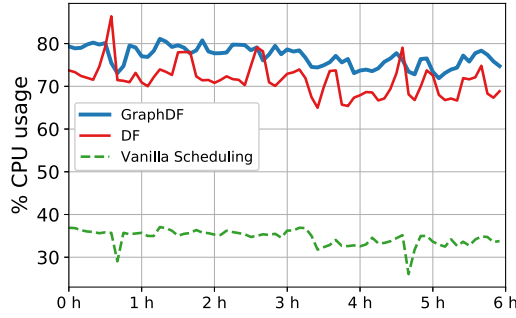


Fig. 7. CPU utilization without opportunistic workload scheduling (shown in green) and with scheduling based on each forecaster (shown in red and blue), over a period of 6 hours on Google dataset. GraphDF-based scheduling leads to higher CPU utilization than DF and vanilla scheduling.

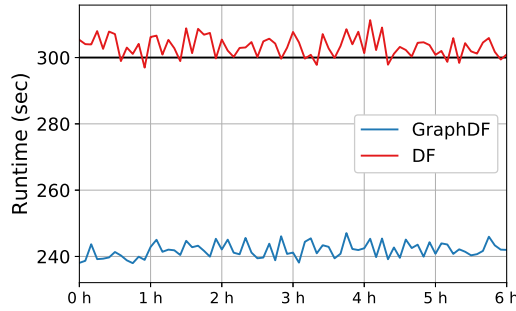


Fig. 8. The time constraint (black line), the runtime of the scheduler with DF (red line), and that with GraphDF (blue line). Note that in most cases, DF fails to meet the time constraint while GraphDF produces a forecast much faster.

cloud efficiency through effective usage forecasting. Google dataset receive observations every 5 minutes, therefore, DF will fail as it is too slow.

Opportunistic Scheduling with IOGraphDF. Using the same parameter setup as in previous section, we further conducted opportunistic scheduling with IOGraphDF model, as shown in Figure 10. We observe that scheduling based on IOGraphDF performs worse than GraphDF at the beginning, which is due to insufficient training of IOGraphDF model; however, over time IOGraphDF give close performance to the GraphDF model. Also, IOGraphDF gives more smooth scheduling curve than GraphDF model. Since IOGraphDF delivers more accurate prediction over

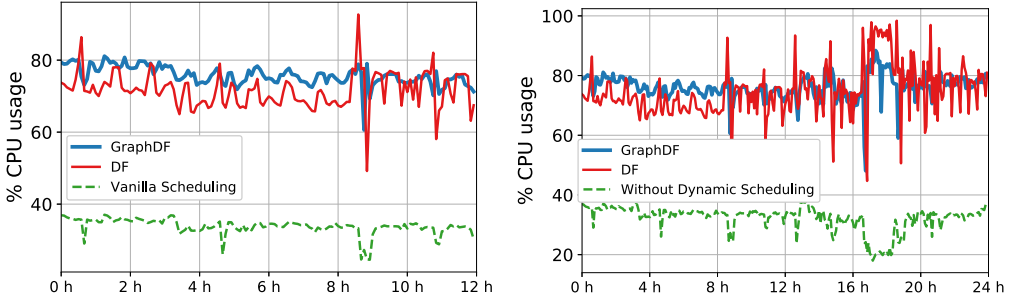


Fig. 9. Using the Google dataset, we simulate a scheduler over 12 hours and 24 hours (144, 288 timestamps). The lag is selected as 6 and the horizon is set as 3. The figure shows improvements over baseline (no forecaster) on CPU utilization by using DF and GraphDF as forecasters. For nodes whose average predicted CPU usage in the horizon are lower than a threshold of 20%, the scheduler will assign tasks that consume 80% CPU resources to the nodes. The overall average utilization improvement of DF is 37.8%, 43.8%. The overall average utilization improvement of GraphDF is 41.9%, 42.6%.

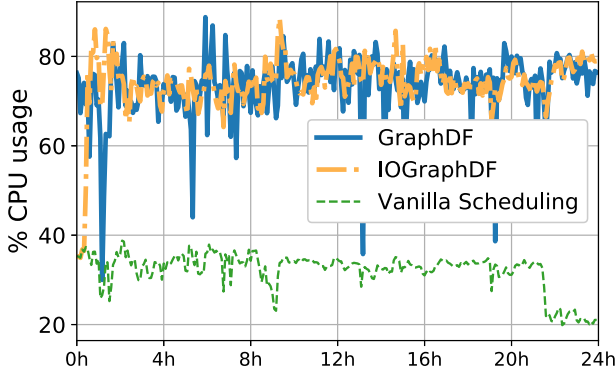


Fig. 10. CPU utilization without opportunistic workload scheduling (shown in green) and with scheduling based on each forecaster (shown in blue and yellow), over a period of 6 hours on Google dataset.

Table 13. Results for Opportunistic Scheduling in the Cloud over a 24 Hour Period using Different Forecasting Models

Data	Model	utilization improvement (%)	correct ratio (%)	cancellation ratio (%)
Google	GraphDF	41.9	88.0	4.6
	IOGraphDF	42.7	90.1	2.9
Adobe	GraphDF	56.3	97.3	0.80
	IOGraphDF	54.3	98.5	0.44

time, the opportunistic scheduling with IOGraphDF also outperforms scheduling with GraphDF model with respect to correct ratio and cancellation ratio, as shown in Table 13.

7 CONCLUSION

In this work, we introduced a deep graph-based probabilistic forecasting framework called GraphDF. While existing deep probabilistic forecasting approaches do not explicitly leverage a

graph, and assume either complete independence among time-series (*i.e.*, completely disconnected graph) or complete dependence between all time-series (*i.e.*, fully connected graph), this work moved beyond these two extreme cases by allowing nodes and their time-series to have arbitrary and explicit weighted dependencies among each other. Such explicit and arbitrary weighted dependencies between nodes and their time-series are modeled as a graph in the proposed framework. Notably, GraphDF consists of a relational global component that learns complex non-linear time-series patterns globally using the structure of the graph to improve both computational efficiency and performance as well as a relational local model that not only considers its individual time-series but the time-series of nodes that are connected in the graph. The experiments demonstrated the effectiveness of the proposed deep graph-based probabilistic forecasting model in terms of its forecasting performance, runtime, and data efficiency. To address the common streaming nature of many time-series prediction applications where values arrive over timesteps, we propose the IOGraphDF model. Experiments show that IOGraphDF outperforms GraphDF regarding forecasting accuracy and runtime.

REFERENCES

- [1] Nesreen K. Ahmed, Amir F. Atiya, Neamat El Gayar, and Hisham El-Shishiny. 2010. An empirical comparison of machine learning models for time series forecasting. *Econometric Reviews* 29, 5–6 (2010), 594–621.
- [2] Alexander Alexandrov, Konstantinos Benidis, Michael Bohlke-Schneider, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, Danielle C. Maddix, Syama Rangapuram, David Salinas, Jasper Schulz, Lorenzo Stella, Ali Caner Turkmen, and Yuyang Wang. 2020. GluonTS: Probabilistic and neural time series modeling in python. *Journal of Machine Learning Research* 21, 116 (2020), 1–6. Retrieved from <http://jmlr.org/papers/v21/19-820.html>.
- [3] Oren Anava, Elad Hazan, Shie Mannor, and Ohad Shamir. 2013. Online learning for time series prediction. In *Proceedings of the 26th Annual Conference on Learning Theory*. Shai Shalev-Shwartz and Ingo Steinwart (Eds.), Vol. 30. PMLR, Princeton, NJ, 172–184. Retrieved from <https://proceedings.mlr.press/v30/Anava13.html>.
- [4] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. 2018. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. arXiv:1803.01271. Retrieved from <https://arxiv.org/abs/1803.01271>.
- [5] Sandy D. Balkin and J. Keith Ord. 2000. Automatic neural network modeling for univariate time series. *International Journal of Forecasting* 16, 4 (2000), 509–515.
- [6] Kasun Bandara, Christoph Bergmeir, and Hansika Hewamalage. 2021. LSTM-MSNet: Leveraging forecasts on sets of related time series with multiple seasonal patterns. *IEEE Transactions on Neural Networks and Learning Systems* 32, 4 (2021), 1586–1599. DOI : <https://doi.org/10.1109/TNNLS.2020.2985720>
- [7] Konstantinos Benidis, Syama Sundar Rangapuram, Valentin Flunkert, Bernie Wang, Danielle Maddix, Caner Turkmen, Jan Gasthaus, Michael Bohlke-Schneider, David Salinas, Lorenzo Stella, et al. 2020. Neural forecasting: Introduction and literature overview. arXiv:2004.10240. Retrieved from <https://arxiv.org/abs/2004.10240>.
- [8] Ricardo Bianchini, Marcus Fontoura, Eli Cortez, Anand Bonde, Alexandre Muzio, Ana-Maria Constantin, Thomas Moscibroda, Gabriel Magalhaes, Girish Bablani, and Mark Russinovich. 2020. Toward ML-centric cloud platforms. *Communications of the ACM* 63, 2 (2020), 50–59.
- [9] Albert Bifet and Ricard Gavaldà. 2009. Adaptive learning from evolving data streams. In *Proceedings of the Advances in Intelligent Data Analysis VIII*. Niall M. Adams, Céline Robardet, Arno Siebes, and Jean-François Boulicaut (Eds.), Springer, Berlin, 249–260.
- [10] Mikolaj Binkowski, Gautier Marti, and Philippe Donnat. 2018. Autoregressive convolutional neural networks for asynchronous time series. In *Proceedings of the International Conference on Machine Learning*. 580–589.
- [11] Gianluca Bontempi, Souhaib Ben Taieb, and Yann-Aël Le Borgne. 2012. Machine learning strategies for time series forecasting. In *Proceedings of the European Business Intelligence Summer School*. Springer, 62–77.
- [12] Sofiane Brahim-Belhouari and Amine Bermak. 2004. Gaussian process for nonstationary time series prediction. *Computational Statistics and Data Analysis* 47, 4 (2004), 705–712.
- [13] R. N. Calheiros, E. Masoumi, R. Ranjan, and R. Buyya. 2015. Workload prediction using ARIMA model and its impact on cloud applications' QoS. *IEEE Transactions on Cloud Computing* 3, 4 (2015), 449–458.
- [14] Defu Cao, Yujing Wang, Juanyong Duan, Ce Zhang, Xia Zhu, Congrui Huang, Yunhai Tong, Bixiong Xu, Jing Bai, Jie Tong, and Qi Zhang. 2020. Spectral temporal graph neural network for multivariate time-series forecasting. *Advances in neural information processing systems* 33 (2020), 17766–17778.
- [15] Rich Caruana and Alexandru Niculescu-Mizil. 2006. An empirical comparison of supervised learning algorithms. In *Proceedings of the 23rd International Conference on Machine Learning*. 161–168.

- [16] Tianqi Chen, Mu Li, Yutian Li, Min Lin, Naiyan Wang, Minjie Wang, Tianjun Xiao, Bing Xu, Chiyuan Zhang, and Zheng Zhang. 2015. Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems. arXiv:1512.01274. Retrieved from <https://arxiv.org/abs/1512.01274>.
- [17] Y. Cheng, C. Wang, H. Yu, Y. Hu, and X. Zhou. 2019. GRU-ES: Resource usage prediction of cloud workloads using a novel hybrid method. In *Proceedings of the 2019 IEEE 21st International Conference on High Performance Computing and Communications; IEEE 17th International Conference on Smart City; IEEE 5th International Conference on Data Science and Systems*. 1249–1256.
- [18] Junyoung Chung, Caglar Gulcehre, Kyunghyun Cho, and Yoshua Bengio. 2014. Empirical evaluation of gated recurrent neural networks on sequence modeling. In *NIPS 2014 Workshop on Deep Learning, December 2014*.
- [19] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. 2017. Clipper: A low-latency online prediction serving system. In *Proceedings of the NSDI*. 613–627.
- [20] Michael J. Crawley. 2013. Mixed-effects models. *The R Book Second Edition*. 681–714.
- [21] Chirag Deb, Fan Zhang, Junjing Yang, Siew Eang Lee, and Kwok Wei Shah. 2017. A review on time series forecasting techniques for building energy consumption. *Renewable and Sustainable Energy Reviews* 74 (2017), 902–924.
- [22] Christina Delimitrou and Christos Kozyrakis. 2014. Quasar: Resource-efficient and QoS-aware cluster management. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems*. Association for Computing Machinery, New York, NY, 127–144.
- [23] Jinliang Deng, Xiusi Chen, Renhe Jiang, Xuan Song, and Ivor W. Tsang. 2021. ST-norm: Spatial and temporal normalization for multi-variate time series forecasting. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 269–278.
- [24] Ilias Dimoukas, Peyman Mazidi, and Lars Herre. 2019. Neural networks for GEFCom2017 probabilistic load forecasting. *International Journal of Forecasting* 35, 4 (2019), 1409–1423.
- [25] Yuntao Du, Jindong Wang, Wenjie Feng, Sinno Pan, Tao Qin, Renjun Xu, and Chongjun Wang. 2021. Adarnn: Adaptive learning and forecasting of time series. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. 402–411.
- [26] Chenyou Fan, Yuze Zhang, Yi Pan, Xiaoyue Li, Chi Zhang, Rong Yuan, Di Wu, Wensheng Wang, Jian Pei, and Heng Huang. 2019. Multi-horizon time series forecasting with temporal attention learning. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2527–2535.
- [27] W. Fang, Z. Lu, J. Wu, and Z. Cao. 2012. RPPS: A novel resource prediction and provisioning scheme in cloud data center. In *Proceedings of the 2012 IEEE 9th International Conference on Services Computing*. 609–616.
- [28] F. Farahnakian, P. Liljeberg, and J. Plosila. 2013. LiRCUP: Linear regression based CPU usage prediction algorithm for live migration of virtual machines in data centers. In *Proceedings of the 2013 39th Euromicro Conference on Software Engineering and Advanced Applications*. 357–364.
- [29] Fahimeh Farahnakian, Tapio Pahikkala, Pasi Liljeberg, Juha Plosila, and Hannu Tenhunen. 2015. Utilization prediction aware VM consolidation approach for green cloud computing. In *Proceedings of the 2015 IEEE 8th International Conference on Cloud Computing*. IEEE, 381–388.
- [30] Robert Fildes, Michèle Hibon, Spyros Makridakis, and Nigel Meade. 1998. Generalising about univariate forecasting methods: Further empirical evidence. *International Journal of Forecasting* 14, 3 (1998), 339–358.
- [31] Valentin Flunkert, David Salinas, and Jan Gasthaus. 2017. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. arXiv:1704.04110. Retrieved from <https://arxiv.org/abs/1704.04110>.
- [32] Everette S. Gardner. 1985. Exponential smoothing: The state of the art. *Journal of Forecasting* 4 (1985), 1–28.
- [33] Andrew Gelman, John B. Carlin, Hal S. Stern, David B. Dunson, Aki Vehtari, and Donald B. Rubin. 2013. *Bayesian Data Analysis*. CRC Press.
- [34] Xu Geng, Yaguang Li, Leye Wang, Lingyu Zhang, Qiang Yang, Jieping Ye, and Yan Liu. 2019. Spatiotemporal multi-graph convolution network for ride-hailing demand forecasting. *Proceedings of the AAAI Conference on Artificial Intelligence* 33 (2019).
- [35] Amir Ghaderi, Borhan M. Sanandaji, and Faezeh Ghaderi. 2017. Deep forecast: Deep learning-based spatio-temporal forecasting. arXiv:1707.08110. Retrieved from <https://arxiv.org/abs/1707.08110>.
- [36] Agathe Girard, Carl Edward Rasmussen, Joaquin Quinonero Candela, and Roderick Murray-Smith. 2003. Gaussian process priors with uncertain inputs application to multiple-step ahead time series forecasting. In *Proceedings of the Advances in Neural Information Processing Systems*. 545–552.
- [37] Tian Guo, Zhao Xu, Xin Yao, Haifeng Chen, Karl Aberer, and Koichi Funaya. 2016. Robust online time series prediction with recurrent neural networks. In *Proceedings of the 2016 IEEE International Conference on Data Science and Advanced Analytics*. IEEE, 816–825.
- [38] Andrew C. Harvey. 1990. *Forecasting, Structural Time Series Models and the Kalman Filter*. Cambridge University Press.

- [39] T. Hastie, R. Tibshirani, and J. H. Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer.
- [40] Mikael Henaff, Junbo Zhao, and Yann LeCun. 2017. Prediction under uncertainty with error-encoding networks. arXiv:1711.04994. Retrieved from <https://arxiv.org/abs/1711.04994>.
- [41] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural Computation* (1997).
- [42] Siteng Huang, Donglin Wang, Xuehan Wu, and Ao Tang. 2019. DSANet: Dual self-attention network for multivariate time series forecasting. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*. Association for Computing Machinery, New York, NY, 2129–2132.
- [43] Thomas N. Kipf and Max Welling. 2016. Variational graph auto-encoders. *NIPS Workshop on Bayesian Deep Learning* (2016).
- [44] Thomas N. Kipf and Max Welling. 2017. Semi-supervised classification with graph convolutional networks. In *Proceedings of the International Conference on Learning Representations*.
- [45] Jitendra Kumar and Ashutosh Kumar Singh. 2018. Workload prediction in cloud using artificial neural network and adaptive differential evolution. *Future Generation Computer Systems* 81 (2018), 41–52.
- [46] Nikolay Laptev, Jason Yosinski, Li Erran Li, and Slawek Smyl. 2017. Time-series extreme event forecasting with neural networks at uber. In *Proceedings of the International Conference on Machine Learning*, Vol. 34. 1–5.
- [47] Vincent Le Guen and Nicolas Thome. 2020. Probabilistic time series forecasting with shape and temporal diversity. *Advances in Neural Information Processing Systems* 33 (2020).
- [48] Xuerong Li, Wei Shang, and Shouyang Wang. 2019. Text-based crude oil price forecasting: A deep learning approach. *International Journal of Forecasting* 35, 4 (2019), 1548–1560.
- [49] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. 2018. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. In *Proceedings of the International Conference on Learning Representations*.
- [50] Bryan Lim. 2018. Forecasting treatment responses over time using recurrent marginal structural networks. In *Proceedings of the Advances in Neural Information Processing Systems*. 7483–7493.
- [51] Bryan Lim, Stefan Zohren, and Stephen Roberts. 2019. Enhancing time-series momentum strategies using deep neural networks. *The Journal of Financial Data Science* 1, 4 (2019), 19–38.
- [52] Chenghao Liu, Steven C. H. Hoi, Peilin Zhao, and Jianling Sun. 2016. Online arima algorithms for time series prediction. In *Proceedings of the 13th AAAI Conference on Artificial Intelligence*.
- [53] Danielle C. Maddix, Yuyang Wang, and Alex Smola. 2018. Deep factors with gaussian processes for forecasting. arXiv:1812.00098. Retrieved from <https://arxiv.org/abs/1812.00098>.
- [54] Spyros Makridakis and Michele Hibon. 1997. ARMA models and the box–Jenkins methodology. *Journal of Forecasting* 16, 3 (1997).
- [55] Spyros Makridakis, Steven C. Wheelwright, and Rob J. Hyndman. 2008. *Forecasting Methods and Applications*. John Wiley and Sons.
- [56] Shanka Subhra Mondal, Nikhil Sheoran, and Subrata Mitra. 2021. Scheduling of time-varying workloads using reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 9000–9008.
- [57] Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. 2020. N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net. <https://openreview.net/forum?id=r1ecqn4YwB>
- [58] Fotios Petropoulos, Daniele Apiletti, Vassilios Assimakopoulos, Mohamed Zied Babai, Devon K. Barrow, Souhaib Ben Taieb, Christoph Bergmeir, Ricardo J. Bessa, Jakub Bijak, John E. Boylan, Jethro Browell, Claudio Carnevale, Jennifer L. Castle, Pasquale Cirillo, Michael P. Clements, Clara Cordeiro, Fernando Luiz Cyrino Oliveira, Shari De Baets, Alexander Dokumentov, Joanne Ellison, Piotr Fiszeder, Philip Hans Franses, David T. Frazier, Michael Gilliland, M. Sinan Gönül, Paul Goodwin, Luigi Grossi, Yael Grushka-Cockayne, Mariangela Guidolin, Massimo Guidolin, Ulrich Gunter, Xiaojia Guo, Renato Guseo, Nigel Harvey, David F. Hendry, Ross Hollyman, Tim Januschowski, Jooyoung Jeon, Victor Richmond R. Jose, Yanfei Kang, Anne B. Koehler, Stephan Kolassa, Nikolaos Kourentzes, Sonia Leva, Feng Li, Konstantia Litsiou, Spyros Makridakis, Gael M. Martin, Andrew B. Martinez, Sheik Meeran, Theodore Modis, Konstantinos Nikolopoulos, Dilek Önkal, Alessia Paccagnini, Anastasios Panagiotelis, Ioannis Panapakidis, Jose M. Pavia, Manuela Pedio, Diego J. Pedregal, Pierre Pinson, Patricia Ramos, David E. Rapach, J. James Reade, Bahman Rostami-Tabar, Michał Rubaszek, Georgios Sermpinis, Han Lin Shang, Evangelos Spiliotis, Aris A. Syntetos, Priyanga Diliini Talagala, Thiyanga S. Talagala, Len Tashman, Dimitrios Thomakos, Thordis Thorarinsdottir, Ezio Todini, Juan Ramón Trapero Arenas, Xiaoqian Wang, Robert L. Winkler, Alisa Yusupova, and Florian Ziel. 2022. Forecasting: theory and practice. *International Journal of Forecasting* 38, 3 (2022), 705–871. <https://doi.org/10.1016/j.ijforecast.2021.11.001>
- [59] Yao Qin, Dongjin Song, Haifeng Chen, Wei Cheng, Guofei Jiang, and Garrison W. Cottrell. 2017. A Dual-Stage Attention-Based Recurrent Neural Network for Time Series Prediction. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI 2017, Melbourne, Australia, August 19-25, 2017*, Carles Sierra (Ed.). ijcai.org, 2627–2633. <https://doi.org/10.24963/ijcai.2017/366>

- [60] Syama Sundar Rangapuram, Matthias W. Seeger, Jan Gasthaus, Lorenzo Stella, Yuyang Wang, and Tim Januschowski. 2018. Deep state space models for time series forecasting. In *Proceedings of the Advances in Neural Information Processing Systems*. 7785–7794.
- [61] Md. Rasheduzzaman, Md. Amirul Islam, and Rashedur M. Rahman. 2014. Workload prediction on Google cluster trace. *International Journal of Grid and High Performance Computing* 6, 3 (2014), 34–52.
- [62] Kashif Rasul, Abdul-Saboor Sheikh, Ingmar Schuster, Urs Bergmann, and Roland Vollgraf. 2020. Multivariate probabilistic time series forecasting via conditioned normalizing flows. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net. <https://openreview.net/forum?id=WiQBfVVRv>
- [63] Cédric Richard, José Carlos M. Bermudez, and Paul Honeine. 2008. Online prediction of time series data with kernels. *IEEE Transactions on Signal Processing* 57, 3 (2008), 1058–1067.
- [64] Ryan A. Rossi. 2018. Relational time series forecasting. *Knowledge Engineering Review* 33 (2018), e1.
- [65] David Salinas, Michael Bohlke-Schneider, Laurent Callot, Roberto Medico, and Jan Gasthaus. 2019. High-dimensional multivariate forecasting with low-rank Gaussian copula processes. In *Proceedings of the Advances in Neural Information Processing Systems*. 6824–6834.
- [66] Rajat Sen, Hsiang-Fu Yu, and Inderjit S. Dhillon. 2019. Think globally, act locally: A deep neural network approach to high-dimensional time series forecasting. In *Proceedings of the Advances in Neural Information Processing Systems*. 4837–4846.
- [67] Youngjoo Seo, Michaël Deerrard, Pierre Vandergheynst, and Xavier Bresson. 2018. Structured Sequence Modeling with Graph Convolutional Recurrent Networks. In *Neural Information Processing - 25th International Conference, ICONIP 2018, Siem Reap, Cambodia, December 13-16, 2018, Proceedings, Part I (Lecture Notes in Computer Science)*, Long Cheng, Andrew Chi-Sing Leung, and Seiichi Ozawa (Eds.). Vol. 11301. Springer, 362–373. https://doi.org/10.1007/978-3-030-04167-0_33
- [68] Zhiming Shen, Sethuraman Subbiah, Xiaohui Gu, and John Wilkes. 2011. Cloudscale: Elastic resource scaling for multi-tenant cloud systems. In *Proceedings of the 2nd ACM Symposium on Cloud Computing*. 1–14.
- [69] James Stock and M. W. Watson. 2001. Vector autoregressions. *Journal of Economic Perspectives* (2001).
- [70] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. 2014. Sequence to sequence learning with neural networks. In *Proceedings of the Advances in Neural Information Processing Systems 27*. Z. Ghahramani, M. Welling, C. Cortes, N. D. Lawrence, and K. Q. Weinberger (Eds.), Curran Associates, Inc.
- [71] Souhaib Ben Taieb, Antti Sorjamaa, and Gianluca Bontempi. 2010. Multiple-output modeling for multi-step-ahead time series forecasting. *Neurocomputing* 73 (2010), 1950–1957.
- [72] Shivaram Venkataraman, Aurojit Panda, Ganesh Ananthanarayanan, Michael J. Franklin, and Ion Stoica. 2014. The power of choice in data-aware cluster scheduling. In *Proceedings of the OSDI*. 301–316.
- [73] Petra Vrablecová, Viera Rozinajová, and Anna Bou Ezzeddine. 2017. Incremental adaptive time series prediction for power demand forecasting. In *Proceedings of the International Conference on Data Mining and Big Data*. Springer, 83–92.
- [74] Bao Wang, Xiyang Luo, Fangbo Zhang, Baichuan Yuan, Andrea L. Bertozzi, and P. Jerrey Brantingham. 2018. Graph-Based Deep Modeling and Real Time Forecasting of Sparse Spatio-Temporal Data. CoRR abs/1804.00684 (2018). arXiv:1804.00684 <http://arxiv.org/abs/1804.00684>.
- [75] Yuyang Wang, Alex Smola, Danielle C. Maddix, Jan Gasthaus, Dean Foster, and Tim Januschowski. 2019. Deep Factors for Forecasting. In *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA (Proceedings of Machine Learning Research)*, Kamalika Chaudhuri and Ruslan Salakhutdinov (Eds.). Vol. 97. PMLR, 6607–6617. <http://proceedings.mlr.press/v97/wang19k.html>.
- [76] Ruofeng Wen, Kari Torkkola, Balakrishnan (Murali) Narayanaswamy, and Dhruv Madeka. 2017. A multi-horizon quantile recurrent forecaster. In *NeurIPS 2017*. <https://www.amazon.science/publications/a-multi-horizon-quantilerecurrent-forecaster>.
- [77] Peter R. Winters. 1960. Forecasting sales by exponentially weighted moving averages. *Management Science* 6, 3 (1960), 324–342.
- [78] Sifan Wu, Xi Xiao, Qianggang Ding, Peilin Zhao, WEI Ying, and Junzhou Huang. 2020. Adversarial sparse transformer for time series forecasting. *Advances in Neural Information Processing Systems* 33 (2020), 17105–17115.
- [79] Jingqi Yang, Chuanchang Liu, Yanlei Shang, Bo Cheng, Zexiang Mao, Chunhong Liu, Lisha Niu, and Junliang Chen. 2014. A cost-aware auto-scaling approach using the workload prediction in service clouds. *Information Systems Frontiers* 16, 1 (2014), 7–18.
- [80] Huaxiu Yao, Xianfeng Tang, Hua Wei, Guanjie Zheng, and Zhenhui Li. 2019. Revisiting spatial-temporal similarity: A deep learning framework for traffic prediction. In *Proceedings of the 2019 AAAI Conference on Artificial Intelligence*.
- [81] Huaxiu Yao, Fei Wu, Jintao Ke, Xianfeng Tang, Yitian Jia, Siyu Lu, Pinghua Gong, Jieping Ye, and Li Zhenhui. 2018. Deep multi-view spatial-temporal network for taxi demand prediction. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*.

- [82] Bing Yu, Haoteng Yin, and Zhanxing Zhu. 2018. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence*.
- [83] G. Peter Zhang and Min Qi. 2005. Neural network forecasting for seasonal and trend time series. *European Journal of Operational Research* 160, 2 (2005), 501–514.
- [84] Hang Zhao, Yujing Wang, Juanyong Duan, Congrui Huang, Defu Cao, Yunhai Tong, Bixiong Xu, Jing Bai, Jie Tong, and Qi Zhang. 2020. Multivariate time-series anomaly detection via graph attention network. In *Proceedings of the 2020 IEEE International Conference on Data Mining*. IEEE, 841–850.
- [85] Qazi Zia Ullah, Shahzad Hassan, and Gul Muhammad Khan. 2017. Adaptive resource utilization prediction system for infrastructure as a service cloud. *Computational Intelligence and Neuroscience* 2017 (2017).

Received 6 January 2022; accepted 8 May 2022